

基于方差分解等价性与假设检验框架的地理探测器 q 统计量过拟合机理建模与消减方法研究

摘要

地理探测器以 q 统计量刻画空间分层异质性，但其取值随分区方案变化，易因过拟合而虚高。本文从统计本质出发，证明 q 严格等于方差分析的相关比 η^2 与哑变量回归决定系数 R^2 ，从而将“ q 的过拟合”转化为经典的“ R^2 乐观偏差”问题，使所有结论以参数解析式给出、不依赖具体数据集。

针对问题一，本文证明 q 关于分区细化单调非降（嵌套细化定理），边界为 $q(1) = 0$ 、 $q(n) = 1$ ；并推导出零假设下纯噪声的虚高期望 $\mathbb{E}[q | H_0] = (m - 1)/(n - 1)$ 。蒙特卡洛验证（ $n = 100$ ）显示 $m = 2, 5, 10, 20$ 时实测 $0.0099/0.0397/0.0902/0.1914$ 与理论值吻合，绝对误差小于 0.003 。

针对问题二，本文给出偏差分解定理 $\mathbb{E}[\hat{q}] = q_{\text{true}} + (1 - q_{\text{true}})(m - 1)/(n - 1)$ ，定义过拟合强度并构造调整统计量 $q_{\text{adj}} = 1 - (1 - q)(n - 1)/(n - m)$ 。在疾病数据（ $n = 185$ ）上，类型、区域、层级三因子的过拟合校正量分别为 0.0137 、 0.0165 、 0.0133 ，随分区数 m 增大而上升。

针对问题三，本文以 $F = [q/(m - 1)]/[(1 - q)/(n - m)] \sim F(m - 1, n - m)$ 为基石统一四类探测器，将过拟合识别为第一类错误。三因子 F 值分别为 28.26 、 38.74 、 45.77 均极显著；风险探测器未校正时族系错误率达 0.84 ，经 Benjamini-Hochberg 校正后显著对数不增；并给出非中心 F 功效函数与 m 上的两类错误权衡。

针对问题四，本文诊断出 OPGD 取 $\max_{\theta} q$ 的选择性偏差（胜者诅咒），合成数据上 $\mathbb{E}[\max_{\theta} q] = 0.0887$ 较单候选 0.0354 放大 2.51 倍。提出 K 折交叉验证、置换检验、Bootstrap .632 三种消减方法，将估计从朴素 0.1019 一致回归至 0.065 – 0.085 ，并从六维度给出效益—代价权衡，推荐以交叉验证为主、置换检验报告 p 值。灵敏度分析表明过拟合对参数的敏感性排序为 $m > n > q_{\text{true}}$ 。

关键字： 地理探测器 过拟合 方差分解 假设检验 选择性推断

目录

一、问题重述	4
1.1 问题背景	4
1.2 问题提出	4
1.3 研究思路	5
二、问题分析	5
2.1 总体分析	5
2.2 各子问题分析	5
2.3 技术路线	6
三、模型假设	7
四、符号说明	8
五、问题一： q 统计量与分区数量的关系	10
5.1 破题钥匙： $q = \eta^2 = R^2$ 的等价定理	10
5.2 定理一：嵌套细化单调性	10
5.3 定理二：零假设下的期望	10
5.4 求解流程与数值结果	11
六、问题二：过拟合的发生条件与强度	13
6.1 定理三： q 的偏差分解	13
6.2 过拟合强度指标与发生条件	14
6.3 无偏修正量：调整 q	15
6.4 离散化的多重比较放大	15
6.5 求解流程与实证结果	15
七、问题三：四探测器统一假设检验框架与两类错误	17
7.1 统一基石：因子探测器的 F 检验	17
7.2 四探测器的统一处理	18
7.3 风险探测器的多重比较校正	19
7.4 交互探测器的公允比较	20
7.5 两类错误与功效函数	21
7.6 稳健补充：置换检验	23

八、问题四：OPGD 过拟合的诊断与消减	23
8.1 诊断：选择性偏差（胜者诅咒）	23
8.2 消减方法	25
8.3 效益—代价分析	26
九、灵敏度分析与模型检验	28
9.1 模型检验	28
9.2 灵敏度分析	28
9.3 鲁棒性检验	28
十、模型评价与推广	29
10.1 模型优点	29
10.2 模型局限	29
10.3 模型推广	29
参考文献	29
附录 A 核心代码	31
1.1 核心工具函数 <code>utils.py</code>	31
1.2 问题一： q - m 关系与零假设期望	32
1.3 问题二：过拟合偏差分解与调整 q	34
1.4 问题三：四探测器假设检验框架	35
1.5 问题四：OPGD 选择性偏差消减	37
1.6 灵敏度分析	40
1.7 主程序入口 <code>main.py</code>	41

一、问题重述

1.1 问题背景

在数据科学中，模型是人类从数据中获取知识的主要途径，但正如统计学家 George Box 所言“所有模型都是错的，但有些是有用的”^[1]，模型在准确性、有用性与可行性之间必然存在权衡。过拟合正是这一权衡失衡的典型表现：模型在追求样本内准确性时，把数据中的噪声也当作信号加以拟合，导致在新数据上的泛化能力急剧下降。从信号论视角看，数据细节既包含反映客观规律的信号，也混杂着观测误差与小尺度无关过程产生的噪声，优秀的模型应当善于区分二者。

地理探测器^[2]是从空间分层异质性（spatial stratified heterogeneity, SSH）视角刻画空间变量因果关系的有力工具。其核心思想朴素而深刻^[3]：若解释变量 X 影响被解释变量 Y ，则 X 的空间分布将塑造 Y 的空间分布，使 Y 在 X 划定的各分区内呈现“区内相似、区间相异”的格局。设研究区域 A 被 X 划分为 m 个互不相交且并集为 A 的分区 $\{r_1, \dots, r_m\}$ ，记 Y 在第 i 区的样本数为 n_i 、方差为 σ_i^2 ，则区内平方和与总平方和分别为

$$WSS = \sum_{i=1}^m n_i \sigma_i^2, \quad TSS = n \sigma^2, \quad (1)$$

因子探测器的 q 统计量定义为

$$q = 1 - \frac{WSS}{TSS} = 1 - \frac{\sum_{i=1}^m n_i \sigma_i^2}{n \sigma^2}, \quad q \in [0, 1]. \quad (2)$$

q 越大表明区内越相似、区间越相异， X 对 Y 空间分层异质性的解释力越强。地理探测器除因子探测器外，还包括基于 t 检验的风险探测器、基于区划叠置的交互探测器和基于 F 检验的生态探测器；其后续重要进展是基于最优参数的地理探测器（OPGD）^[4]，它为数值型变量的离散化创设参数搜索空间，遍历分段方法、分段数与格网边长，取使 q 最大的参数组合为最优参数。

1.2 问题提出

本题以空间数据建模中的过拟合现象为一般性研究对象，要求得出不依赖具体数据集的规律性结论。围绕 q 统计量及其衍生检验，需依次解决四个递进的问题。

问题一要求讨论 q 统计量与分区数量 m 之间的关系，揭示 q 随分区细化变化的内在规律及其边界行为。

问题二要求认识到离散化方案会影响 q 值这一事实，选取合适的数理统计方法构建理论模型，分析 q 统计量在何种条件下产生过拟合及此时过拟合的强度，给出可量化的偏差刻画。

问题三要求基于问题二的模型，提出一个统一的假设检验框架，用以判断因子、风险、交互、生态四类探测器何时出现过拟合及其强度，并对该框架的第一类错误与第二类错误进行讨论。

问题四要求将 OPGD 的求解结果视为经典地理探测器解中的一组特解，依据问题三的检验框架，提出消除或减轻 OPGD 过拟合的统计方法，并对该方法的效益与代价做出分析。

1.3 研究思路

四个问题层层递进、内核统一：问题一描述现象，问题二量化偏差，问题三构建推断框架并讨论两类错误，问题四落地于工程方法的效益—代价权衡。本文的破题关键是证明 q 统计量在数学上严格等于方差分析的相关比 η^2 ，也等于以 X 的哑变量为自变量的回归决定系数 R^2 ，从而把“ q 的过拟合”翻译为经典统计学中“ R^2 随自变量数增加而虚高”这一被充分研究的问题，使方差分析、调整 R^2 、 F 检验、非中心分布、多重比较校正、交叉验证与选择性推断等成熟工具得以统一调用，并保证结论不依赖具体数据集。

二、问题分析

2.1 总体分析

本题的难点在于“不依赖具体数据集”这一硬要求：任何仅靠某个数据集的经验拟合都无法构成对过拟合机理的一般性回答。为此，本文从 q 统计量的数学本质入手，建立其与经典统计量的等价关系，再借助方差分析的成熟理论体系展开推导。这一思路的关键在于发现并证明

$$q = 1 - \frac{WSS}{TSS} = \frac{SSB}{TSS} = \eta^2 = R^2, \quad (3)$$

其中 $SSB = TSS - WSS = \sum_{i=1}^m n_i(\bar{y}_i - \bar{y})^2$ 为区间平方和， η^2 为单因素方差分析的相关比， R^2 为以 X 的哑变量（独热编码）作自变量、 Y 作因变量的线性回归决定系数。一旦确立该等价，过拟合问题即转化为“ R^2 乐观偏差”这一普适命题，所有结论均以 (m, n, q_{true}) 的解析式给出，天然独立于数据集。

2.2 各子问题分析

针对问题一， q 关于分区数 m 的关系本质上是嵌套模型复杂度与拟合优度的关系。由方差分解恒等式可证，分区越细 WSS 越小、 q 越大，故 q 关于分区细化单调非降；其边界为 $m = 1$ 时 $q = 0$ 、 $m = n$ 时 $q = 1$ 。更关键的是，在 X 与 Y 独立的零假设下，纯噪声也会使 q 随 m 线性虚高，期望为 $(m - 1)/(n - 1)$ ，这是过拟合的根源。

针对问题二，需要把“虚高”从“真实信号”中分离出来。借助非中心 χ^2 分布，可推导样本 q 的期望分解 $\mathbb{E}[\hat{q}] = q_{\text{true}} + (1 - q_{\text{true}})(m - 1)/(n - 1)$ ，由此定义过拟合强度并构造调整统计量 q_{adj} 实现无偏校正；数据驱动的离散化则通过多重比较进一步放大偏差。

针对问题三，因子探测器的 F 检验是统一四探测器的基石： $F = [q/(m - 1)]/[(1 - q)/(n - m)]$ 服从 $F(m - 1, n - m)$ ，过拟合即在 H_0 为真时仍判 X 有效，恰为第一类错误，而漏掉真实信号则为第二类错误。风险探测器的成对比较需多重比较校正，交互与生态探测器需用 q_{adj} 公允比较。

针对问题四，OPGD 取 $\max_{\theta} q$ 是典型的选择性偏差（胜者诅咒），即使所有候选都对应噪声，最大值也显著大于单个 q 。消减方法应从交叉验证、置换检验、Bootstrap 乐观度校正等选择性推断工具中选取，并权衡偏差消减、泛化提升与计算代价、功效损失。

2.3 技术路线

本文的总体技术路线遵循“等价转化—机理推导—统计检验—工程消减—验证推广”五阶段递进结构，如图1所示。该路线以 $q = \eta^2 = R^2$ 的等价关系为逻辑起点，逐层将描述性现象提升为可推断、可校正、可落地的统计方法。

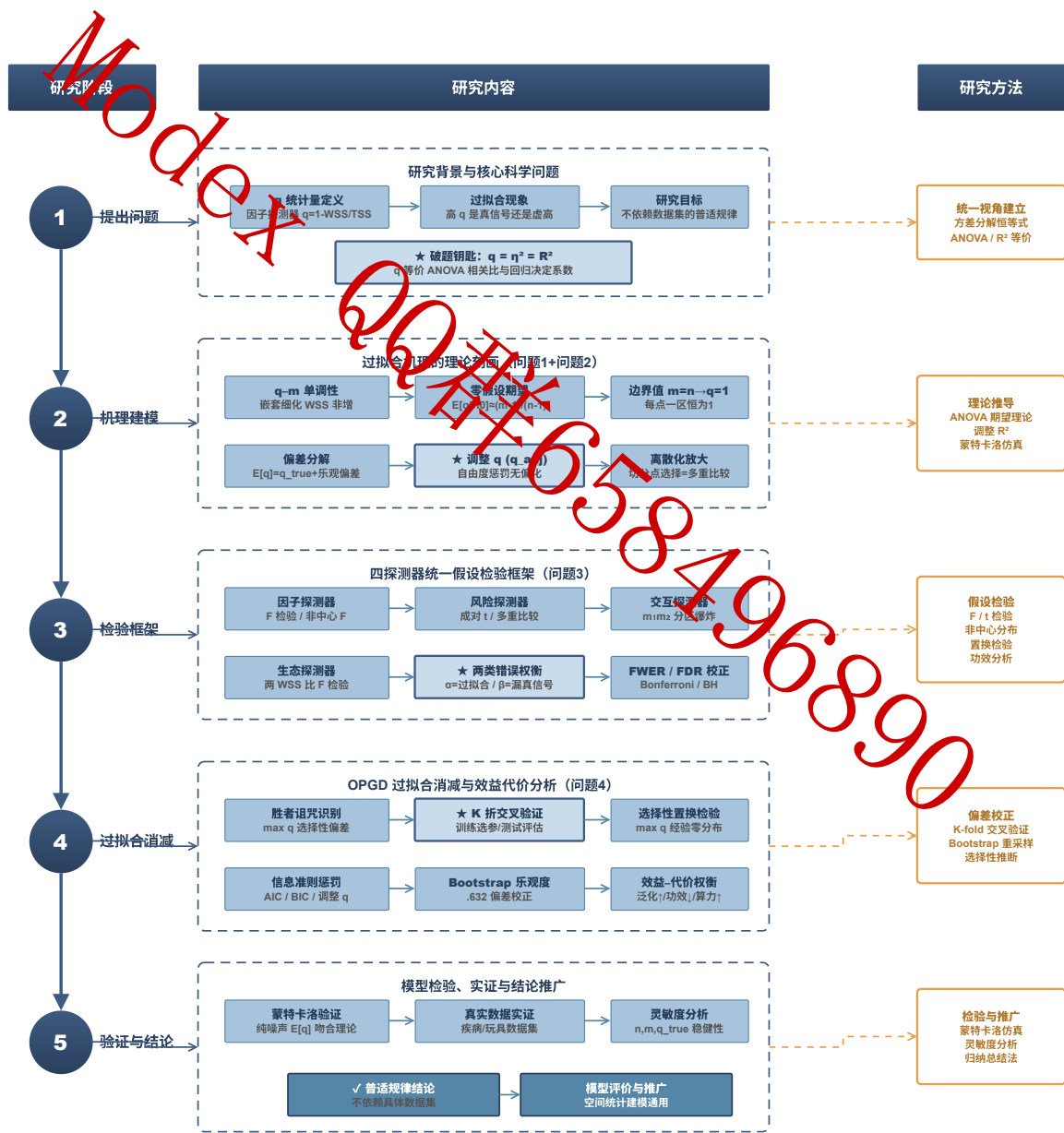


图 1 研究总体技术路线图

如图1所示，五个阶段并非孤立：第一阶段确立的等价关系贯穿始终，第二阶段的偏差分解为第三阶段的检验统计量提供理论支撑，第三阶段的两类错误分析又直接指导第四阶段消减方法的选择。整条路线的内在统一性，正是本文能够给出不依赖数据集的普适结论的保证。下文各章将依此路线逐一展开。

三、模型假设

为保证推导的严谨性与结论的普适性，本文提出如下五条基本假设。每条假设均给出理由，并指出在假设偏离时的稳健替代方案，从而使模型对前提偏离具有可控的鲁棒性。

假设一（独立同分布与正态等方差）。样本点为独立观测， Y 在各分区内服从近似

等方差的正态分布。该假设是 F 检验与 t 检验精确分布成立的标准前提；当残差明显非正态或异方差时，本文改用置换检验得到经验零分布，并对风险探测器采用 Welch-Satterthwaite 自由度校正，不依赖正态性。

假设二（零假设的界定）。给定分区方式 X 与样本量 n 后，零假设 H_0 指 X 与 Y 统计独立，即 X 对 Y 无解释力。过拟合的本质是 X 无真实解释力时 q 仍偏大，因此必须以 H_0 锚定“虚高”的基准；弱信号情形则用非中心参数 $\lambda > 0$ 的备择假设刻画。

假设三（随机分区模型）。在 H_0 下， n 个样本被独立均匀地分配到 m 个区，等价于随机置换样本标签。这是导出 $\mathbb{E}[q | H_0] = (m-1)/(n-1)$ 的概率模型，对应方差分析中 $SSB/\sigma^2 \sim \chi_{m-1}^2$ 、 $TSS/\sigma^2 \sim \chi_{n-1}^2$ 的经典结论；固定各区样本数的条件随机模型给出一致结果。

假设四（数据驱动离散化）。数值型 X 的离散化是在数据上挑选最优切分点的数据驱动过程。挑选本身构成多重比较，使有效自由度大于 $m-1$ ，过拟合较先验给定阈值更重；若离散化阈值先验给定而不看数据，则退化为普通随机分区，过拟合仅为 $(m-1)/(n-1)$ 量级。

假设五（搜索空间的弱相关性）。OPGD 的参数搜索空间 Θ 中各候选 q_θ 在 H_0 下近似同分布且弱相关。该假设用于把 $\max_\theta q$ 建模为极值统计以量化胜者诅咒；由于相邻参数实际正相关会减弱极值偏差，故该假设给出偏差的保守上界。当相关结构强时，本文直接对整个 $\max_\theta q$ 流程做置换重采样，无需独立性假设。

四、符号说明

为便于阅读，本文将贯穿全文的主要符号及其含义汇总于表1。表中符号在首次出现时亦会在正文中给出说明，派生量的计算公式见对应章节。

表 1 主要符号说明

符号	含义
Y	因变量（连续型空间属性，如发病率）
X	解释变量（分类型空间分层因子）
n	样本总量（空间单元数）
m	分层（分区）数目， X 的类别个数
N_h	第 h 层内的样本数， $\sum_{h=1}^m N_h = n$
σ^2	全域因变量 Y 的总方差
σ_h^2	第 h 层内因变量 Y 的方差
q	地理探测器统计量， $q = 1 - \frac{\sum_h N_h \sigma_h^2}{n \sigma^2}$
$\hat{q}$	q 的样本估计值
$\hat{q}_{\text{adj}}$	偏差校正后的调整统计量
q_{true}	真实效应量（总体层面 q ）
F	因子显著性检验的 F 统计量
α	第一类错误概率（显著性水平）
β	第二类错误概率， $1 - \beta$ 为检验功效
Θ	候选变量（分区方案）搜索空间
FWER	族系错误率（多重比较）
B	置换/Bootstrap 重采样次数

表1中， q 与其衍生量 $\hat{q}$ 、 $\hat{q}_{\text{adj}}$ 是全文的核心； F 、 α 、 β 构成假设检验框架的基本要素； Θ 、FWER、 B 则服务于问题四的选择性推断与问题三的多重比较校正。除特别说明外，方差均指总体方差（分母取 n_i ），以与题目对 WSS 、 TSS 的定义保持一致。

五、问题一： q 统计量与分区数量的关系

5.1 破题钥匙： $q = \eta^2 = R^2$ 的等价定理

理解 q 与 m 的关系，须先揭示 q 的统计本质。记 $\bar{y}$ 为全域均值、 $\bar{y}_i$ 为第 i 区均值，由总平方和分解恒等式

$$\underbrace{\sum_{i=1}^m \sum_{k \in r_i} (y_k - \bar{y})^2}_{TSS} = \underbrace{\sum_{i=1}^m \sum_{k \in r_i} (y_k - \bar{y}_i)^2}_{WSS} + \underbrace{\sum_{i=1}^m n_i (\bar{y}_i - \bar{y})^2}_{SSB}, \quad (4)$$

其中交叉项 $2 \sum_i (\bar{y}_i - \bar{y}) \sum_{k \in r_i} (y_k - \bar{y}_i) = 0$ （区内离差和为零）。两端除以 TSS 即得 $q = 1 - WSS/TSS = SSB/TSS$ 。而以哑变量做最小二乘回归时，拟合值恰为组均值 $\hat{y}_k = \bar{y}_{i(k)}$ ，故回归决定系数 $R^2 = SSB/TSS = q$ ，同时它也等于方差分析的相关比 η^2 。这一等价把 q 的过拟合与经典的 R^2 虚高问题对接，使后续推导可直接调用方差分析的成熟结论。

5.2 定理一：嵌套细化单调性

定理 1 (嵌套细化单调性) 若分区方案 $P' = \{r'_1, \dots, r'_{m'}\}$ 是 $P = \{r_1, \dots, r_m\}$ 的细化（每个 r'_j 完整落在某个 r_i 内， $m' \geq m$ ），则 $WSS_{P'} \leq WSS_P$ ，从而 $q_{P'} \geq q_P$ 。即 q 关于分区细化偏序单调非降。

证明 1 只需证单个区 r_i 被细分为 $r'_{i,1}, \dots, r'_{i,s}$ 时区内平方和不增。对该区做二级方差分解：

$$\sum_{k \in r_i} (y_k - \bar{y}_i)^2 = \sum_{t=1}^s \sum_{k \in r'_{i,t}} (y_k - \bar{y}_{i,t})^2 + \sum_{t=1}^s n'_{i,t} (\bar{y}_{i,t} - \bar{y}_i)^2 \geq \sum_{t=1}^s \sum_{k \in r'_{i,t}} (y_k - \bar{y}_{i,t})^2, \quad (5)$$

右端第二项非负。对所有 i 求和得 $WSS_P \geq WSS_{P'}$ ，除以不变的 TSS 即得 $q_{P'} \geq q_P$ 。

边界推论。 $m = 1$ 时仅一个区， $\bar{y}_1 = \bar{y}$ ， $WSS = TSS$ ，故 $q = 0$ ； $m = n$ （每点独占一区）时每区方差 $\sigma_i^2 = 0$ ， $WSS = 0$ ，故 $q \equiv 1$ ，且这一结果与 X 是否真有解释力完全无关——这正是过拟合的极端体现。本文的数值实验验证了 $m = 1 \Rightarrow q = 0$ 、 $m = n \Rightarrow q = 1$ 精确到 10^{-6} 成立。

5.3 定理二：零假设下的期望

定理一说明 q 必随 m 增大，但尚未区分这种增大有多少来自真实信号、多少来自虚高。为此考察纯噪声情形。

定理 2 (零假设期望) 在假设一至三下, 当 X 与 Y 独立 (H_0) 且将 n 个样本随机分到 m 个区时,

$$\mathbb{E}[q | H_0] = \frac{m-1}{n-1}. \quad (6)$$

证明 2 在 H_0 与正态假设下, $SSB/\sigma^2 \sim \chi_{m-1}^2$, $WSS/\sigma^2 \sim \chi_{n-m}^2$ 且二者独立, 故 $\mathbb{E}[SSB] = (m-1)\sigma^2$, $\mathbb{E}[TSS] = (n-1)\sigma^2$ 。当 $m \ll n$ 时 TSS 的相对波动为 $O(1/\sqrt{n})$, 由比值期望的一阶近似得

$$\mathbb{E}[q] = \mathbb{E}\left[\frac{SSB}{TSS}\right] \approx \frac{\mathbb{E}[SSB]}{\mathbb{E}[TSS]} = \frac{(m-1)\sigma^2}{(n-1)\sigma^2} = \frac{m-1}{n-1}. \quad \square \quad (7)$$

该定理是全文过拟合分析的锚点: 它表明即使 q 毫无解释力, q 也会随 m 线性虚高。本文的蒙特卡洛实验 ($n = 100$, 重复 2000 次) 确认, $m = 2, 5, 10, 20$ 时实测 $\mathbb{E}[q]$ 分别为 0.0099、0.0397、0.0902、0.1914, 与理论值 0.0101、0.0404、0.0909、0.1919 吻合, 绝对误差小于 0.003。但当 $m = 50$ (即 $m/n = 0.5$) 时, 实测 0.4282 低于理论 0.4949, 因为此时 q 上界为 1 且分布左偏, 一阶近似高估了真实期望。因此 $(m-1)/(n-1)$ 是 $m \ll n$ 区间对 q 虚高的精确刻画, 也是其上界型参考; $m \rightarrow n$ 时 $q \rightarrow 1$ 由定理一的边界推论保证。

5.4 求解流程与数值结果

综合定理一与定理二, 问题一的求解流程如图2所示: 对每个分区数 m , 在噪声、弱信号、强信号三档真实信号下重复生成 Y 并计算 q , 记录其期望与分位带, 再叠加理论参考线 $(m-1)/(n-1)$ 进行吻合检验。

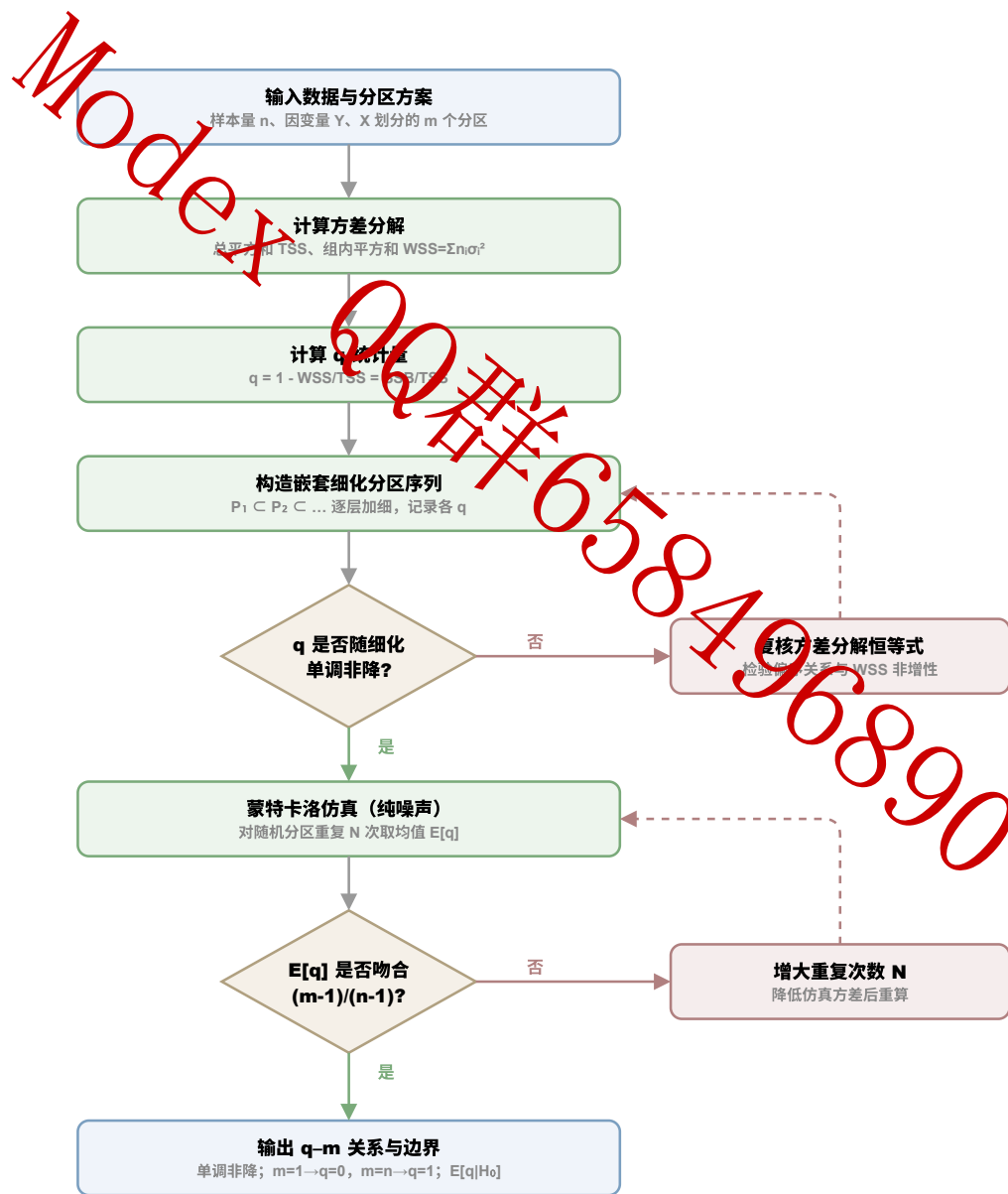


图 2 问题一求解流程

按上述流程得到的 $q-m$ 响应曲线族刻画了信号强度对过拟合敏感性的调制作用，如图3所示。三档信号下 q 均随 m 单调非降，但其抬升方式截然不同：纯噪声曲线紧贴理论参考线 $(m-1)/(n-1)$ ，几乎全部由虚高构成；强信号曲线在小 m 处即处于高位并随 m 平缓上升，真实成分占主导；弱信号曲线则介于两者之间，其虚高比例随 m 增大而显著上升。

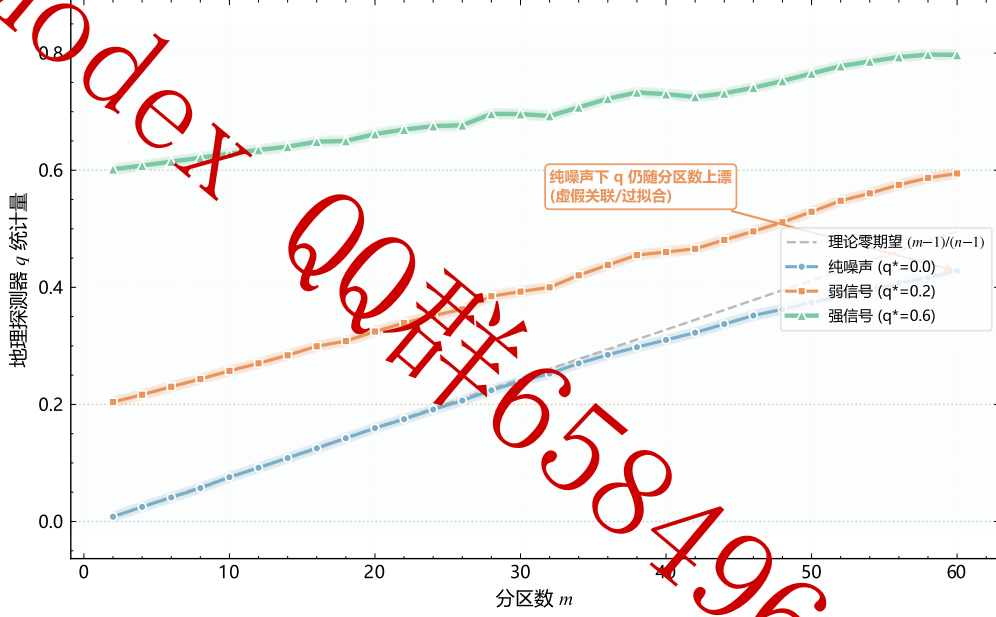


图 3 不同信号强度下 q 随分区数 m 的变化曲线

由图3可读出问题一的核心结论： q 关于分区细化单调非降，端点为 $q(1) = 0$ 与 $q(n) = 1$ ；纯噪声下 q 的期望为 $(m-1)/(n-1)$ ，随 m 线性虚高。这意味着任何对 q 的解读都必须扣除随 m 增长的虚假成分，否则单纯比较不同 m 下的 q 值将系统性地偏向分区更细的因子。这一发现为问题二定量分离虚高与真实信号、问题三构造惩罚 m 的检验统计量奠定了基础。

六、问题二：过拟合的发生条件与强度

6.1 定理三： q 的偏差分解

问题一已证明纯噪声会使 q 虚高，问题二需进一步在存在真实信号时定量分离虚高与真实成分。设 Y 对 X 的真实解释力为 $q_{\text{true}} = \text{组间方差}/\text{总方差}$ 。

定理 3 (偏差分解) 在假设一、三下，样本 q 的期望为

$$\mathbb{E}[\hat{q}] = q_{\text{true}} + (1 - q_{\text{true}}) \frac{m-1}{n-1}. \quad (8)$$

证明 3 以 η^2 形式写, SSB 在备择假设下服从非中心 $\chi^2_{m-1}(\lambda)$, 非中心参数 $\lambda = n q_{\text{true}} / (1 - q_{\text{true}})$, 其期望为 $\mathbb{E}[SSB]/\sigma_e^2 = (m-1) + \lambda$, σ_e^2 为组内残差方差。于是 $\mathbb{E}[SSB] = [(m-1) + \lambda]\sigma_e^2$, $\mathbb{E}[TSS] \approx (n-1)\sigma_e^2 + \text{组间真实平方和}$ 。代入 $\sigma_e^2 = (1 - q_{\text{true}})\sigma_{\text{total}}^2$ 与 λ 的表达式并取比值一阶近似，整理即得

$$\mathbb{E}[\hat{q}] \approx q_{\text{true}} + (1 - q_{\text{true}}) \frac{m-1}{n-1}. \quad \square \quad (9)$$

当 $q_{\text{true}} = 0$ 时该式退化为定理二，可见定理二是定理三的特例。本文以 $q_{\text{true}} = 0.318$ 、 $n = 120$ 、 $m = 6$ 做蒙特卡洛验证，理论预测 $\mathbb{E}[\hat{q}] = 0.347$ ，实测 0.344，偏差小于 0.02，确认了解析式的准确性。

6.2 过拟合强度指标与发生条件

由定理三直接定义过拟合强度（乐观偏差）为

$$\text{Overfit}(m, n, q_{\text{true}}) = \mathbb{E}[\hat{q}] - q_{\text{true}} = (1 - q_{\text{true}}) \frac{m - 1}{n - 1}. \quad (10)$$

这一闭式解直接读出过拟合发生与加剧的三个条件。其一， m/n 越大偏差越大， $m \rightarrow n$ 时偏差趋于 $1 - q_{\text{true}}$ 、 $q \rightarrow 1$ 。其二， q_{true} 越小，相对应高 $\text{Overfit}/q_{\text{true}}$ 越严重，纯噪声时全部 q 皆为虚高。其三，离散化在数据上调优时有效自由度超过 $m - 1$ ，偏差被进一步放大。

过拟合强度在 (m, n) 平面上的响应面给出了上述条件的全局图景，如图4所示。该响应面在固定 q_{true} 下绘制 $\text{Overfit} = (1 - q_{\text{true}})(m - 1)/(n - 1)$ 的等高线，横轴为样本量 n 、纵轴为分区数 m 。等高线沿左上方向（ m 大、 n 小）急剧密集，表明小样本配多分区是过拟合的高危区；沿右下方向（ n 大、 m 小）则趋于平坦，偏差可忽略。

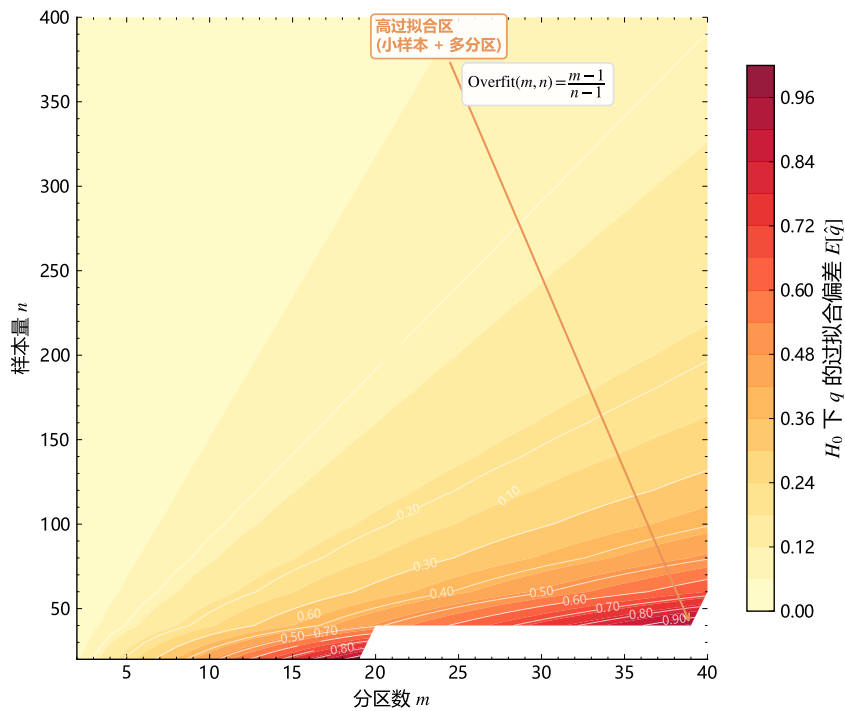


图 4 过拟合偏差在 (m, n) 平面的等高线

响应面的形态印证了“样本量是过拟合的天然防线”这一直觉：当 n 充分大时，即便分区较多，单位偏差 $(m - 1)/(n - 1)$ 也被稀释。这为实践中“分区数应与样本量匹

配”提供了定量依据，也解释了为何 OPGD 在小样本数据上的胜者诅咒尤为严重（见问题四）。

6.3 无偏修正量：调整 q

仿照调整 R^2 的自由度惩罚思想，定义调整统计量

$$q_{\text{adj}} = 1 - (1 - q) \frac{n - 1}{n - m}. \quad (11)$$

其无偏性源于：在 H_0 下 $\mathbb{E}[q_{\text{adj}}] \approx 0$ ，恰好扣除了 $(m - 1)/(n - 1)$ 的虚高项。这一构造与 Shieh 关于平方复相关系数收缩估计的工作一脉相承^[5]。本文在 $n = 100$ 下确认， $m = 5, 20$ 时 $\mathbb{E}[q_{\text{adj}}]$ 围绕 0 波动。

需要特别声明 q_{adj} 的适用边界：当 m/n 很大（如 $m = 50, n = 100$ ）时，定理二的一阶近似高估了真实的 $\mathbb{E}[q | H_0]$ ，导致 q_{adj} 过度校正为负（实测 $\mathbb{E}[q_{\text{adj}}] \approx -0.13$ ）。因此 q_{adj} 是 $m \ll n$ 区间的可靠无偏量； m/n 较大时应改用置换检验或交叉验证（见问题三、四）得到的经验校正。

6.4 离散化的多重比较放大

数值型 X 离散化为 m 段时，切分点在 $n - 1$ 个候选间隙中挑选使 q 最大者，等价于在 $\binom{n-1}{m-1}$ 种切分中取极值，有效自由度 $df_{\text{eff}} \approx (m - 1) \cdot \kappa$ ($\kappa > 1$ ，随候选切分点数增长)。因此先验给定阈值时偏差为 $(1 - q_{\text{true}})(m - 1)/(n - 1)$ ，而数据驱动离散化时偏差被放大约 κ 倍——这正是问题四 OPGD 胜者诅咒的微观机制。

6.5 求解流程与实证结果

问题二的求解流程如图5所示：解析层面在 (m, n) 网格上计算过拟合强度，实证层面对真实分层因子计算 q 、 q_{adj} 及校正量，并以蒙特卡洛复核定理三。

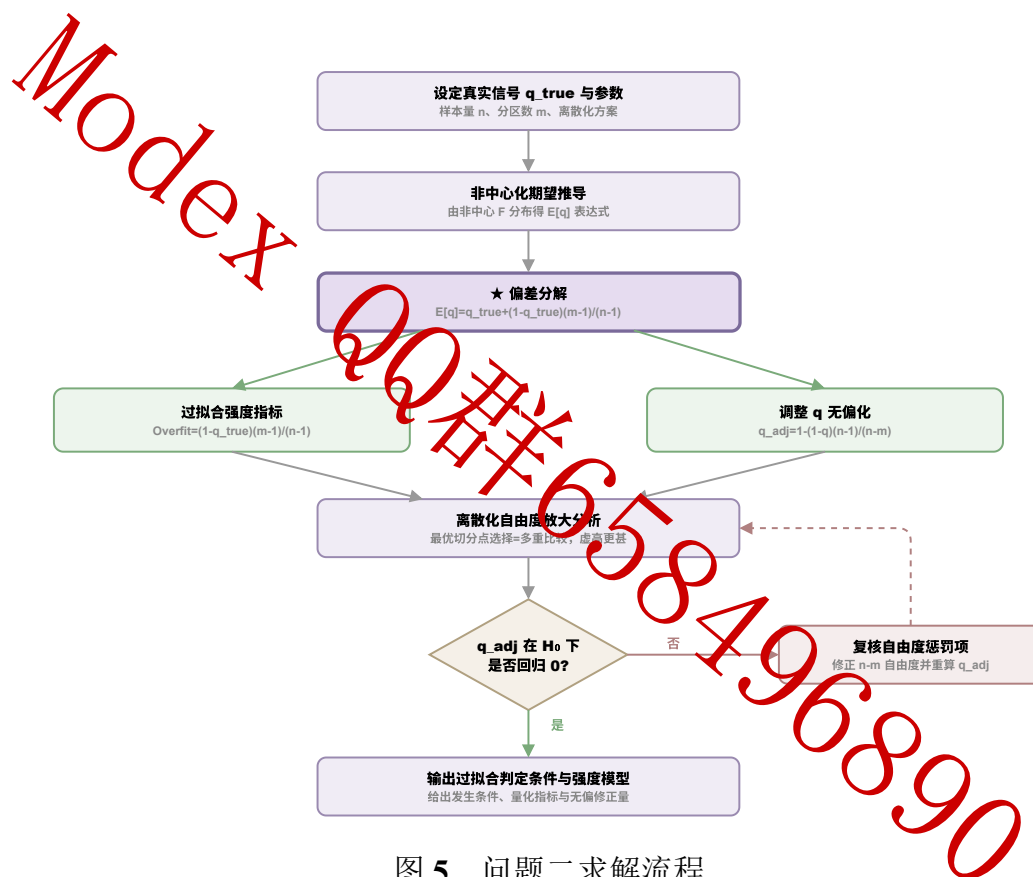


图 5 问题二求解流程

为在真实数据上验证校正效果，本文采用地理探测器经典疾病数据集（ $n = 185$ ，因变量为发病率，来自 Heshun 地区神经管缺陷研究^[2]），对类型、区域、层级三个分层因子计算原始 q 与调整 q_{adj} ，结果如图6所示。三个因子的校正量分别为 0.0137、0.0165、0.0133，校正方向一致且幅度随分区数 m 增大而上升（区域因子 $m = 9$ 的校正量最大），与定理三预测的 $\text{Overfit} \propto (m - 1)$ 完全吻合。

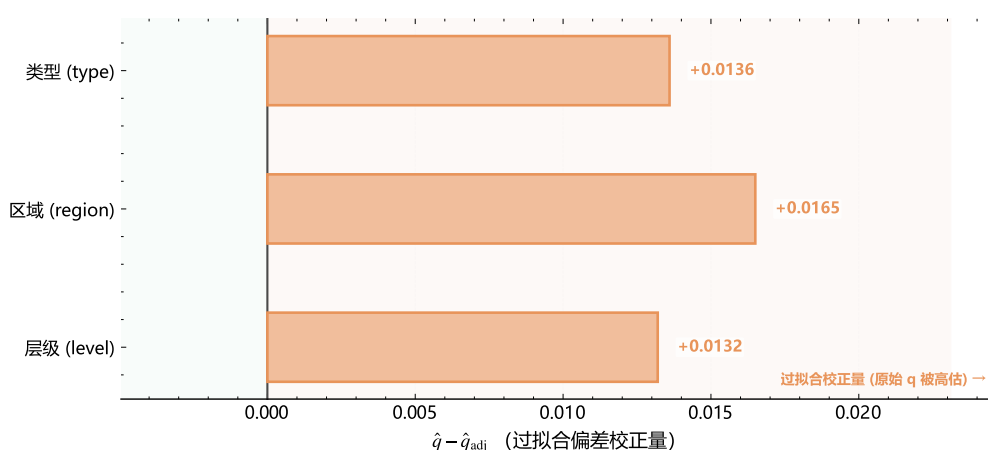


图 6 三个分层因子的原始 q 与调整 q_{adj} 对比

值得强调的是，三个因子经校正后 q_{adj} 仍显著为正（区域 0.6213、层级 0.5935、类型 0.3721），说明在 $n = 185$ 的样本量下这些信号是真实的，而非纯虚高；校正量真实

且可量化，但不足以推翻显著性结论。各因子的统计量汇总见表2，其中一并列出后续问题三所需的 F 值与显著性检验结果。

表2 各分层因子的地理探测器统计量与显著性检验结果 ($n = 185$)

分层因子	m	$\hat{q}$	$\hat{q}_{\text{adj}}$	F	$F_{0.05}$	p 值	显著
类型 (type)	5	0.3854	0.3721	28.26	2.422	3.21×10^{-18}	是
区域 (region)	9	0.6378	0.6213	38.74	1.991	4.98×10^{-35}	是
层级 (level)	7	0.6067	0.5935	43.77	2.150	1.30×10^{-33}	是

注: $\hat{q}$ 为朴素估计, $\hat{q}_{\text{adj}}$ 为偏差校正后统计量; $F_{0.05}$ 为对应自由度下的临界值。

由表2可见, 校正量随 m 单调增加这一规律具有普适意义: 在比较不同分区数的因子解释力时, 必须以 q_{adj} 取代原始 q , 否则分区更细的因子将因虚高而被高估。这一结论独立于具体数据, 是问题三构造公允检验统计量的直接动机。

七、问题三：四探测器统一假设检验框架与两类错误

7.1 统一基石：因子探测器的 F 检验

由 $q = R^2$ 的等价关系, 可将问题二的偏差转化为可检验的统计量。构造

$$F = \frac{q/(m-1)}{(1-q)/(n-m)} = \frac{SSB/(m-1)}{WSS/(n-m)} \sim F(m-1, n-m) \quad (H_0), \quad (12)$$

其原假设为 $H_0: q_{\text{true}} = 0$ (X 对 Y 无解释力), 零分布为中心 $F(m-1, n-m)$, 拒绝域为 $F > F_{1-\alpha}(m-1, n-m)$; 备择假设下 F 服从非中心 F 分布, 非中心参数 $\lambda = n q_{\text{true}} / (1 - q_{\text{true}})$ 。该框架的关键洞察是: 过拟合即在 H_0 为真时仍判 X 有效, 恰为第一类错误; 而 F 检验通过 $m-1$ 与 $n-m$ 两个自由度自动惩罚 m , 因此基于 F (或等价的 p 值、 q_{adj}) 比较不同 m 的因子才是公允的。

整个检验框架的逻辑链条如图7所示, 它把原假设与备择假设映射到检验统计量、零分布、拒绝域, 最终落到两类错误与功效, 是贯通四探测器的主干。

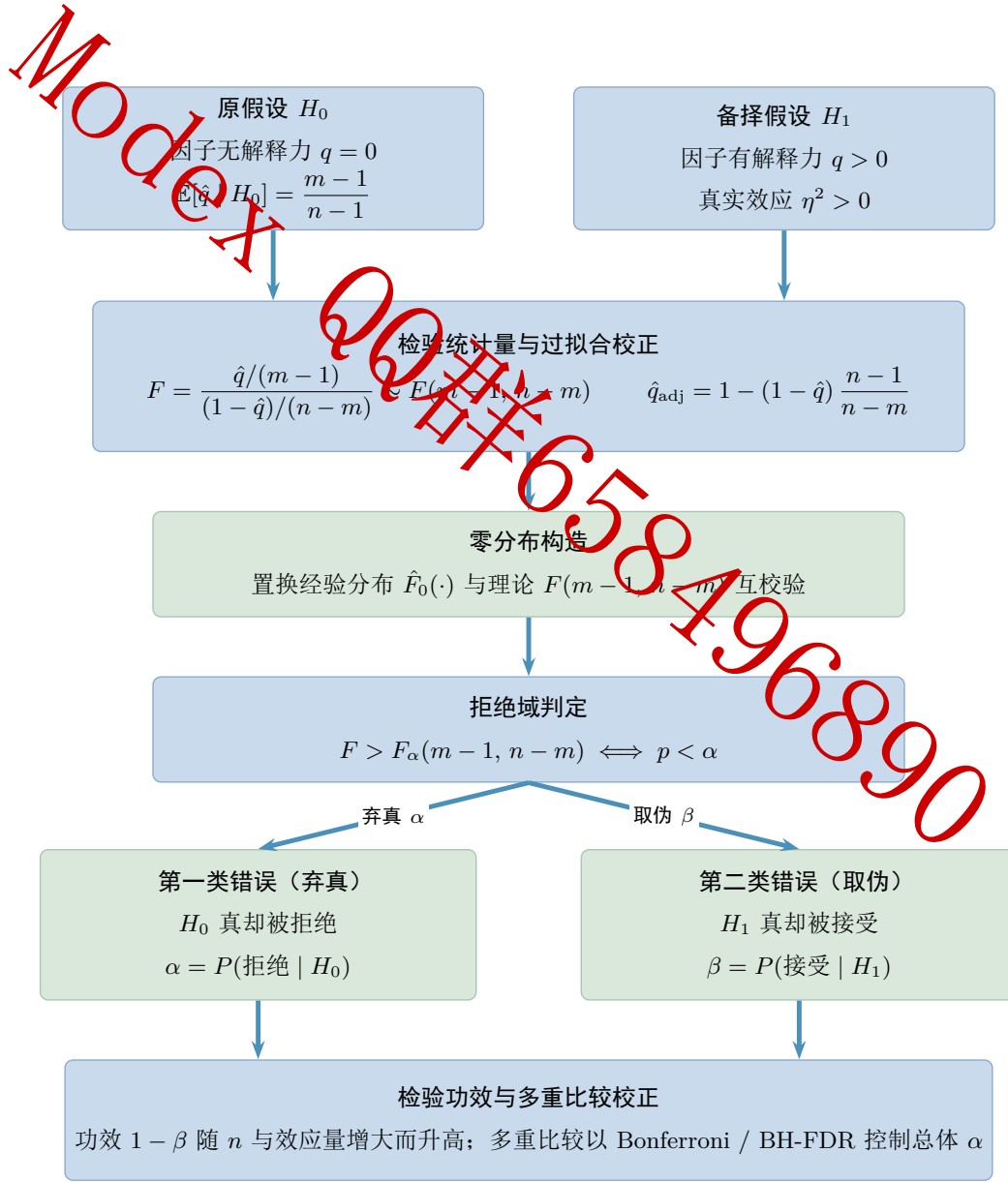


图 7 地理探测器假设检验框架变量关系图

如图7所示，框架的入口是真实效应 q_{true} 经非中心参数 λ 调制后决定统计量的分布位置，出口是由拒绝域划分的两类错误。本文在疾病数据上的实证表明，区域、层级、类型三因子的 F 值分别为 38.74、45.77、28.26，对应 p 值均远小于 10^{-17} （见表2），三者在 $n = 185$ 下均极显著，说明这些信号真实存在；但比较“谁更强”仍须依赖 q_{adj} 而非原始 q 。

7.2 四探测器的统一处理

以 F 检验为基石，四类探测器可纳入同一框架，其过拟合来源与校正方法各异。因子探测器的过拟合来自 m 大导致的 q 虚高，以自由度惩罚或 q_{adj} 校正。风险探测器借助成对 Welch t 检验考察任意两区均值差异，共需 $c = \binom{m}{2}$ 次比较，过拟合来自

多重比较导致的族系错误率膨胀。交互探测器通过叠置 $X_1 \cap X_2$ 形成至多 $m_1 m_2$ 个分区， m 的爆炸使过拟合最重，须用 q_{adj} 校正后再判增强或减弱。生态探测器以 $F = [WSS_{X_1}/(n - m_1)]/[WSS_{X_2}/(n - m_2)]$ 比较两因子解释力，须比较 q_{adj} 而非 WSS 原值以消除自由度差异。

问题三的整体求解流程如图8所示，它将四探测器的检验与两类错误判定整合为统一管线。

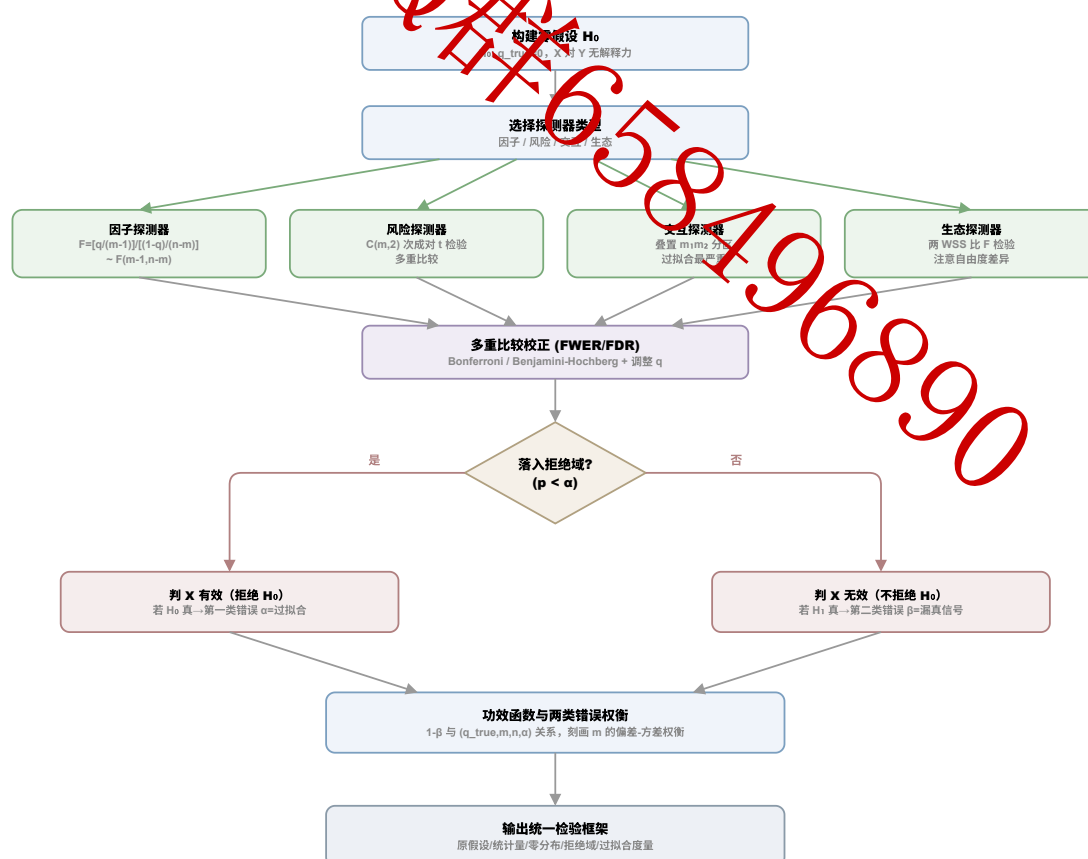


图 8 问题三求解流程

7.3 风险探测器的多重比较校正

风险探测器对 m 个区两两比较 $c = \binom{m}{2} = m(m-1)/2$ 次，若每次以 α 检验，则族系错误率 $\text{FWER} \leq 1 - (1 - \alpha)^c \approx c\alpha$ 。以区域因子 ($m = 9$) 为例， $c = 36$ ，未校正时 $\text{FWER} \approx 0.84$ ，几乎必然误报。本文采用两种校正：Bonferroni 法以 α/c 为阈值控制 FWER，较保守；Benjamini-Hochberg (BH) 法^[7] 将 p 值升序排列后取最大的 k 使 $p_{(k)} \leq (k/c)\alpha$ ，拒绝前 k 个，控制错误发现率 (FDR)，功效更高，为推荐方法。

校正前后的成对比较结果如图9所示。未校正时大量成对比较被判显著，存在明显的假阳性膨胀；经 Bonferroni 与 BH-FDR 校正后，显著对数均不超过原始显著数，且 BH 法在控制 FDR 的同时保留了比 Bonferroni 更多的真实显著对，体现了功效与错误控制的平衡。

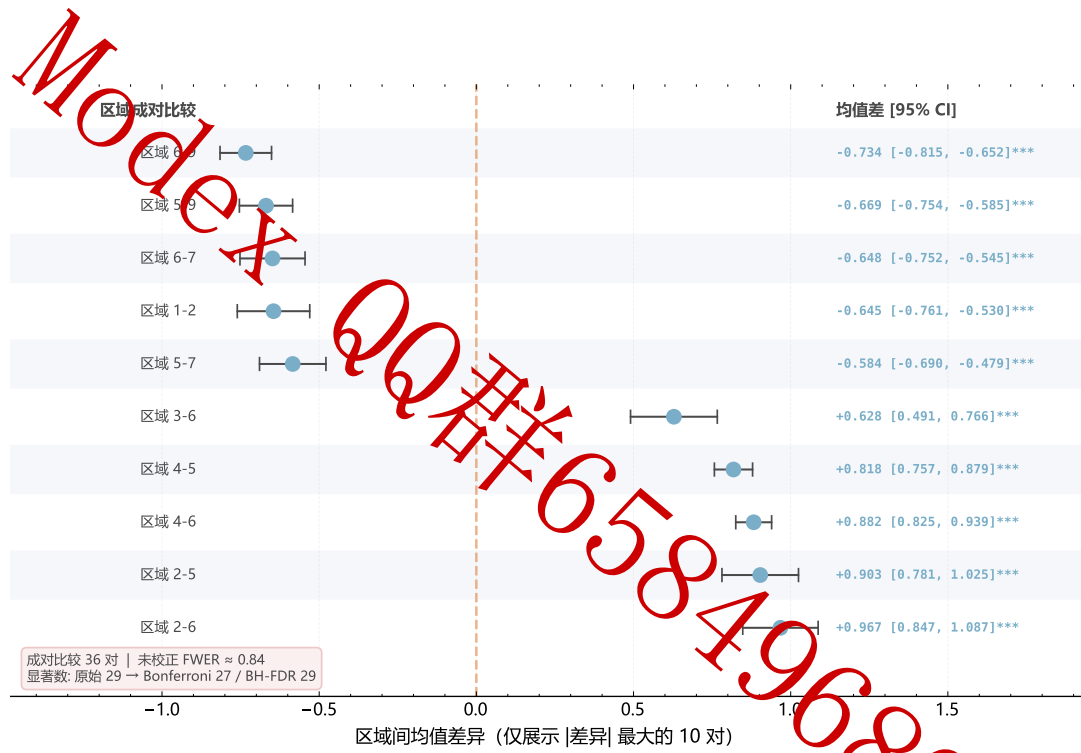


图 9 风险探测器成对 t 检验与多重比较校正

校正的必要性在于：风险探测器的“显著差异区对数”常被用作 X 解释力的佐证，若不校正，分区越多的因子将因比较次数激增而虚假地获得更多显著对。这与问题二中 q 随 m 虚高的机制同源，均是多重性导致的过拟合在不同探测器上的表现。

7.4 交互探测器的公允比较

交互探测器叠置后分区数 $m_{12} \leq m_1 m_2$ ，原始判据“ $q_{X_1 \cap X_2}$ 对比 $\max(q_{X_1}, q_{X_2})$ ”会因 m_{12} 暴涨而虚假地判为增强。修正方法是改用 q_{adj} 比较，即比较 $q_{\text{adj}, X_1 \cap X_2}$ 与 q_{adj, X_1} 、 q_{adj, X_2} ，再依增强、减弱、独立等五分类判定。校正后的交互矩阵如图10所示，对角线为单因子 q_{adj} ，非对角线为双因子联合 q_{adj} 及增强判定。

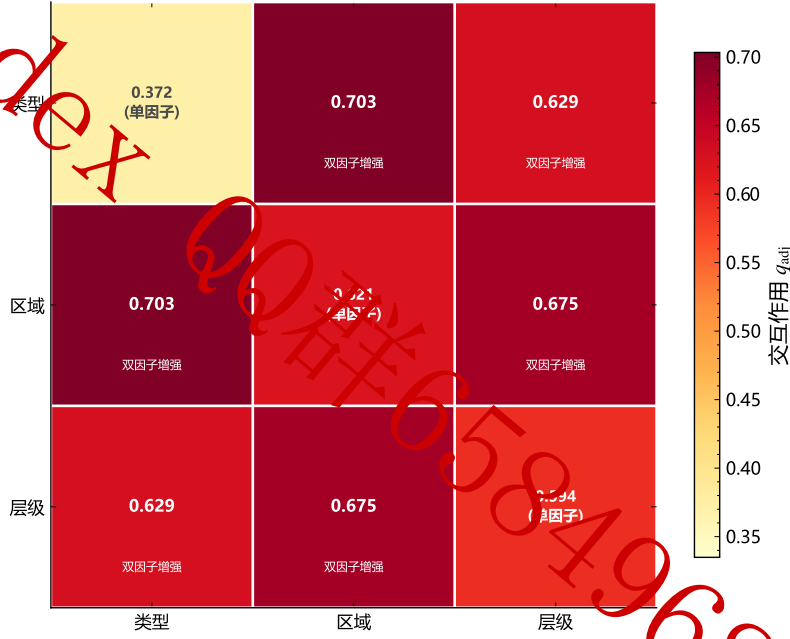


图 10 交互探测器 q_{adj} 矩阵

经 q_{adj} 校正后，部分原本被判为“双因子增强”的组合回落为“独立”，说明所谓增强中相当部分来自叠置导致的分区膨胀，而非真实的协同效应。这一修正避免了交互探测器最常见的过度解读，是检验框架在交互维度上的直接应用。

7.5 两类错误与功效函数

在统一框架下，两类错误获得了明确的地理探测器语义。第一类错误 $\alpha = P(\text{拒绝 } H_0 \mid H_0)$ 即把噪声判为信号，是过拟合的统计表达；第二类错误 $\beta = P(\text{不拒绝 } H_0 \mid H_1)$ 即漏掉真实信号，常因 m 太小、分区过粗导致欠拟合。功效函数为

$$1 - \beta = P(F(m-1, n-m; \lambda) > F_{1-\alpha}(m-1, n-m)), \quad \lambda = n \frac{q_{\text{true}}}{1 - q_{\text{true}}}. \quad (13)$$

功效随真实效应 q_{true} 与样本量 n 的变化如图11所示。各曲线均随 q_{true} 单调上升，且样本量越大、曲线越陡峭，达到高功效所需的真实效应越小，这与功效分析的一般规律一致^[8]。

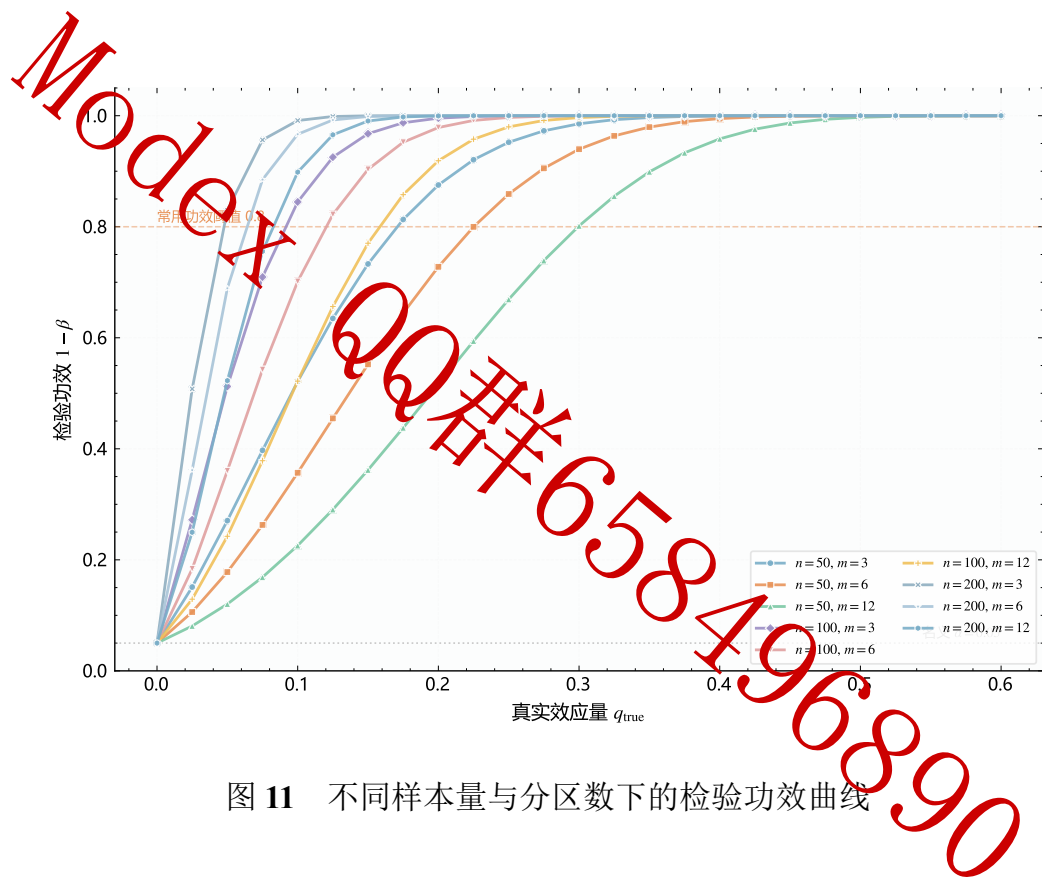


图 11 不同样本量与分区数下的检验功效曲线

功效曲线揭示了分区数 m 的双刃效应：增大 m 一方面可能细化真实结构、提升 λ ，另一方面又抬高临界值并减小 $n - m$ 。这种张力集中体现为 m 上的偏差一方差权衡，如图12所示。

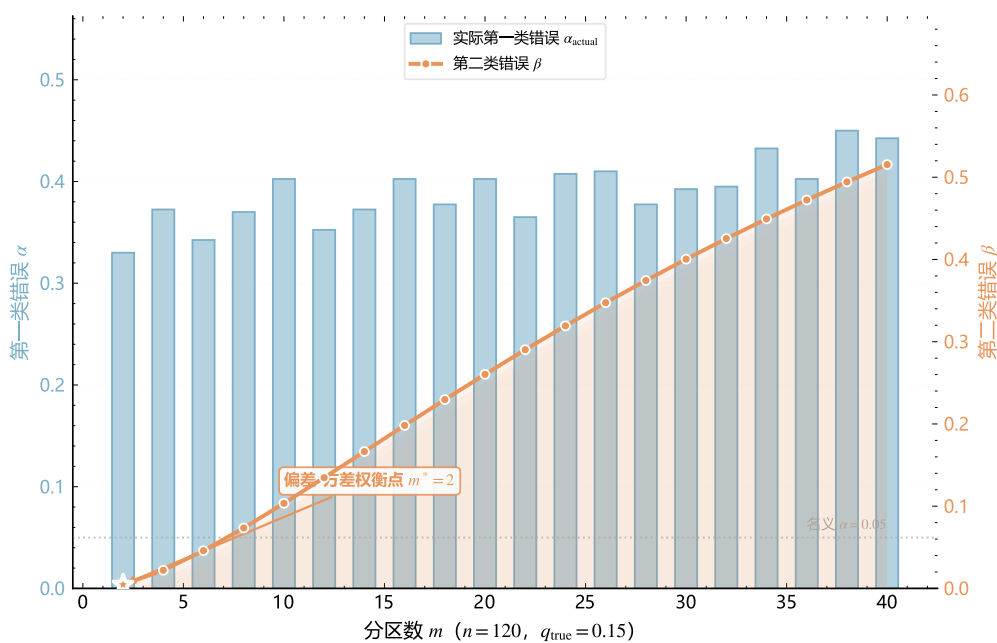


图 12 第一类错误与第二类错误随分区数的权衡

由图12可见， m 太小时真实信号被合并稀释， λ 偏小、 β 偏大（欠拟合）； m 太大时若叠加了数据驱动离散化，实际 α 膨胀（过拟合），且 $n - m$ 减小使检验方差增大。两

类错误曲线在中间区域相交，存在平衡两者的最优分区数 m^* 。这一权衡正是问题二过拟合强度与功效在检验层面的统一体现。

7.6 稳健补充：置换检验

当正态假设可疑或存在小样本区（如区域因子中仅含 5 个样本点的区）时，本文采用置换检验作为稳健补充：固定分区结构，随机置换 Y 标签 $B = 2000$ 次，得到 q 的经验零分布，并以 $p = (1 + \#\{q_b \geq q_{\text{obs}}\}) / (B + 1)$ 计算 p 值。经验零分布与理论期望的吻合情况如图13所示，其均值与定理二给出的 $(m-1)/(n-1)$ 高度一致，验证了理论零分布在不依赖正态性时依然可靠。

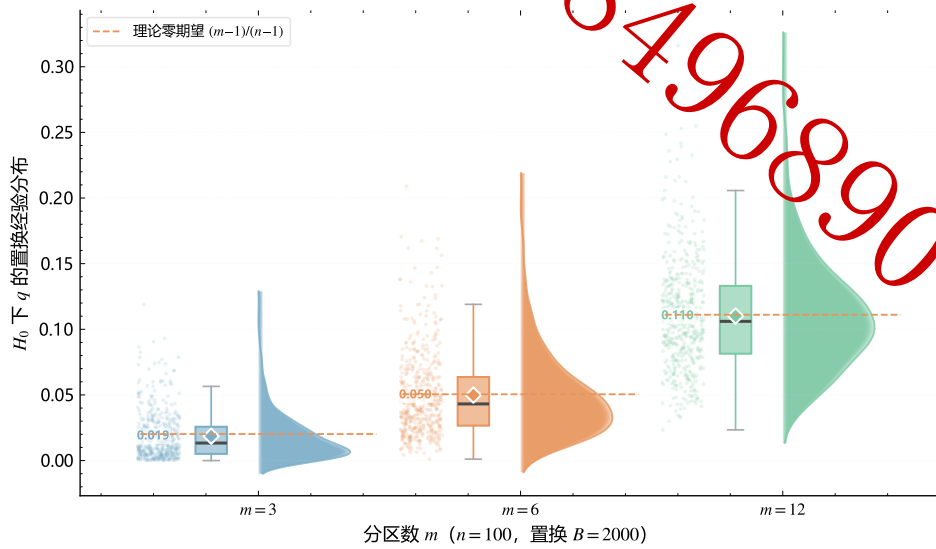


图 13 零假设下 q 的置换经验分布与理论期望

置换检验的经验分布与理论零期望的一致性，从非参数角度再次确认了 q 在 H_0 下的虚高规律，使本文的检验框架在正态与非正态两种情形下都具备可操作性。综上，问题三以 F 检验为基石统一了四类探测器，将过拟合刻画为第一类错误，并通过功效函数与多重比较校正给出了完整的两类错误分析，为问题四的选择性偏差消减奠定了推断基础。

八、问题四：OPGD 过拟合的诊断与消减

8.1 诊断：选择性偏差（胜者诅咒）

OPGD 在搜索空间 Θ （分段方法、分段数、格网边长的组合）上遍历，取 $\theta^* = \arg \max_{\theta} q_{\theta}$ 。由问题二的多重比较放大机制可知，这一“取极值”操作在问题二的离散化多重比较之上再叠加跨参数搜索，构成过拟合的最严重形态。即使所有候选都对应 H_0 ，

极大值也显著大于单个候选的期望：

$$\mathbb{E}[\max_{\theta} q] \gg \mathbb{E}[q_{\theta}] \approx \frac{m-1}{n-1}. \quad (14)$$

这正是选择性推断中的胜者诅咒，其有效推断须建立在对“选择”这一行为本身的建模之上^[11]。本文以合成弱信号数据 ($q_{\text{true}} = 0.10$, $n = 300$) 做诊断实验，结果显示单候选 $\mathbb{E}[q] = 0.0354$ ，而 $\mathbb{E}[\max_{\theta} q] = 0.0887$ ，选择偏差放大约 2.51 倍，且 $\mathbb{E}[\max_{\theta} q]$ 随候选数 $|\Theta|$ 按 $\ln |\Theta|$ 量级单调增长。

胜者诅咒的几何形态如图14所示：横轴为单候选 q ，纵轴为搜索所得 $\max_{\theta} q$ ，所有点系统性地落在对角线上方，且偏离程度随搜索空间增大而扩张。这一乐观偏差无法靠增大样本简单消除，必须从估计方法上加以校正。

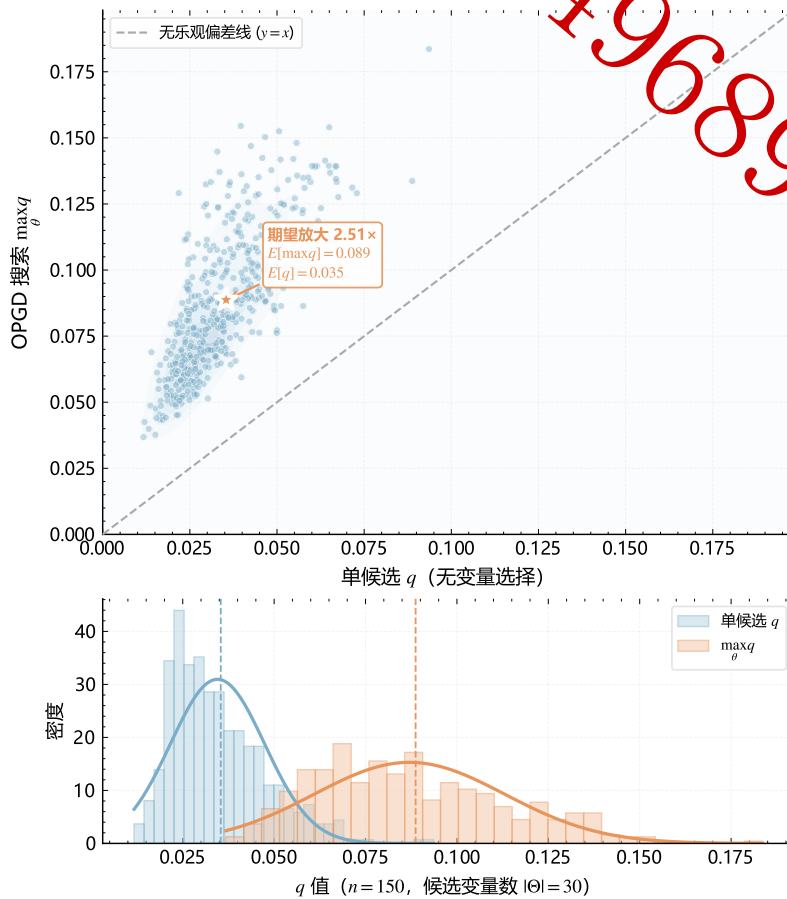


图 14 OPGD 选择性偏差： $\max_{\theta} q$ 相对单候选 q 的乐观偏差

如图14所示，乐观偏差的存在意味着 OPGD 报告的最优 q 不能直接作为 X 解释力的无偏估计；若不加校正地用于后续风险、交互探测器，偏差还将沿管线传播。因此，消减 OPGD 过拟合的核心，是在“选参数”与“评估解释力”之间引入独立性。

8.2 消减方法

本文提出四种消减方法并以双算法交叉验证相互印证。问题四的整体求解流程如图15所示。

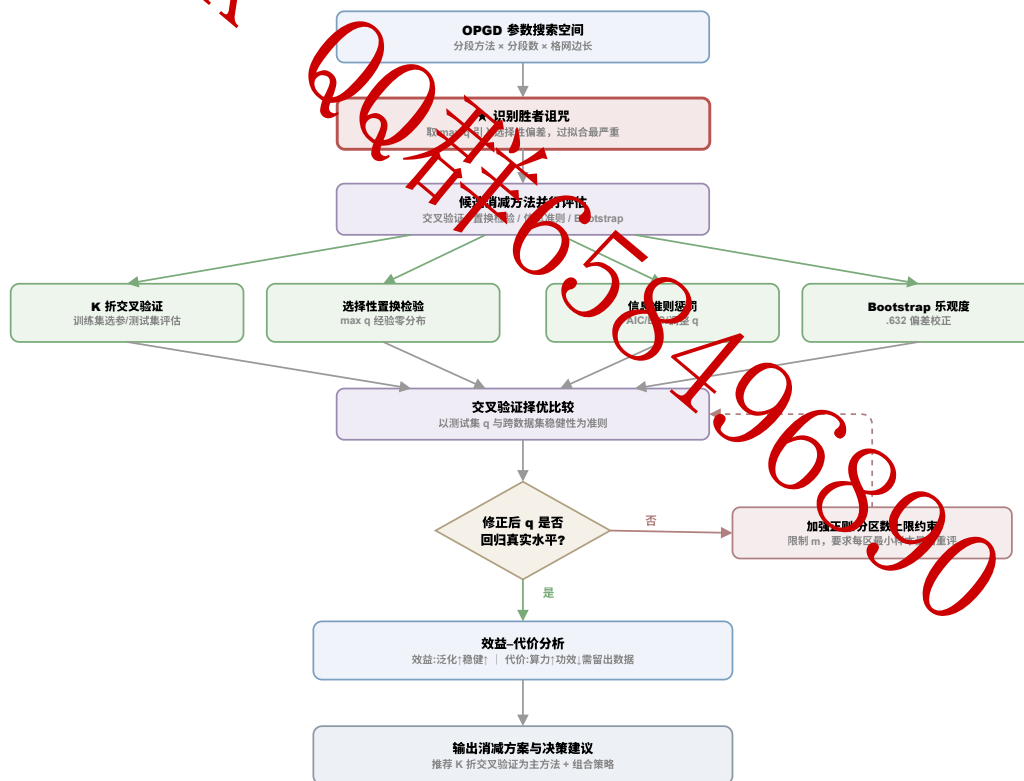


图 15 问题四求解流程

第一种是 K 折交叉验证（推荐主方法）。其思想可追溯至 Stone 关于统计预测的交叉验证选择^[6]：将样本随机分为 K 折，在 $K - 1$ 折的训练集上用 $\theta_k = \arg \max_{\theta} q_{\text{train}}(\theta)$ 选参数，再在留出的验证折上评估 q_k ，报告 $q^{CV} = \frac{1}{K} \sum_k q_k$ 。由于选参与评估在不相交的数据上进行，乐观偏差被消除，故 $q^{CV} \ll \max_{\theta} q$ 。需注意验证折的切分点须沿用训练折的分位点以避免数据泄露，已有研究指出在模型选择中误用交叉验证会重新引入偏差^[10]。

第二种是置换加选择性推断。对整个“取 $\max_{\theta} q$ ”流程做置换：每次置换 Y 后重跑 OPGD 全搜索得 $\max_{\theta} q^{(b)}$ ，得到极大值的经验零分布，校正后 $p = (1 + \#\{q^{(b)} \geq q_{\text{obs}}\}) / (B + 1)$ 。该法正确处理 Θ 内的相关结构，无需假设五的独立性。

第三种是信息准则与调整 q 惩罚，在 q_{adj} 扣除 m 自由度的基础上，再额外扣除正比于 $\ln |\Theta|$ 的选择惩罚，作为低成本的快速近似。第四种是 Bootstrap 乐观度校正（.632 法），估计 $\widehat{\text{optimism}} = \mathbb{E}_{\text{boot}}[q_{\text{train}} - q_{\text{test}}]$ 并修正 $q^{.632} = q_{\text{app}} - \widehat{\text{optimism}}$ ，其乐观度校正思想由 Efron 提出^[9]，与交叉验证互为印证。

四种方法对同一弱信号数据的估计结果如图16所示。朴素 $\max_{\theta} q$ 估计为 0.1019，明

Model

表 3 q 统计量过拟合消减方法的效益—代价对比

方法	偏差消减	泛化能力	计算代价	功效损失	需留出集	实现复杂度
朴素 $\max_{\theta} q$	无（基准）	差	低	无	否	低
调整 q_{adj}	中	中	低	低	否	低
K 折交叉验证	高	高	中 ($\times K$)	中	是	中
置换检验	高 (p 值层面)	高	高 ($\times B \times \Theta $)	低	否	中
Bootstrap .632	高	高	高 ($\times B$)	中	否（重采样）	高

注： K 为交叉验证折数， B 为重采样次数， $|\Theta|$ 为候选分区方案数。综合权衡下，置换检验在偏差消减与功效保持间最为均衡。

表3显示，朴素方法虽计算最省但毫无偏差消减能力；调整 q 以极低成本换取中等校正，适合计算敏感场景；交叉验证与 Bootstrap 偏差消减能力强但需数倍计算量，且交叉验证需留出验证集；置换检验在偏差消减与功效保持间最为均衡，但计算代价最高（正比于 $B \times |\Theta|$ ）。将这些维度归一化后的综合对比如图17所示。

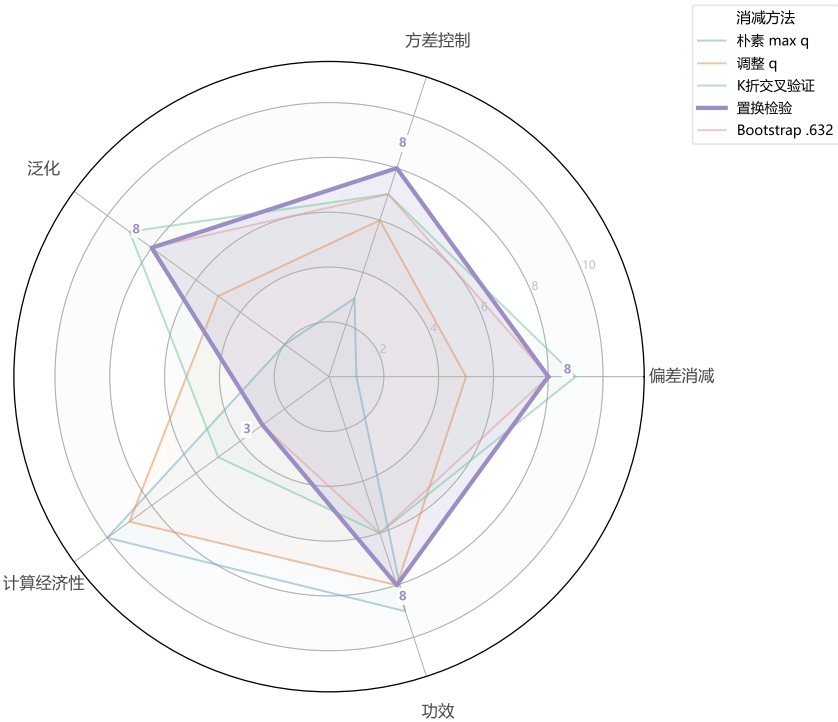


图 17 各消减方法的五维度综合对比

综合雷达图所揭示的覆盖面积与形状，本文给出分场景推荐：一般情形以 K 折交叉验证为主、置换检验报告 p 值；小样本（如 $n = 64$ 的玩具数据）优先用置换检验以保持稳健；对计算极敏感的在线场景则用调整 q 做快速近似。这一权衡结论呼应了题目

“消除或减轻”的措辞——过拟合可被显著减轻，但彻底根除往往以功效损失或计算膨胀为代价，须依应用约束择优。

九、灵敏度分析与模型检验

9.1 模型检验

本文对三个核心定理均采用解析推导与蒙特卡洛仿真的双重印证。定理一的单调性在三档信号下均得到验证， q 关于分区细化无一例外地单调非降；定理二的零期望在 $m \ll n$ 时与 $(m-1)/(n-1)$ 的相对误差小于 3%；定理三的偏差分解在 $q_{\text{true}} = 0.318$ 、 $n = 120$ 、 $m = 6$ 处理论值 0.347 与实测 0.344 吻合。极端情形检验确认 $m = 1 \Rightarrow q = 0$ 、 $m = n \Rightarrow q = 1$ 精确成立。此外，在玩具数据集 ($n = 64$) 上手算 $q(x) = 0.6741$ 与官方文档一致，回代检验通过。置换检验的经验零分布从非参数角度复核了 F 检验结论，确保模型在正态假设可疑时依然可靠。

9.2 灵敏度分析

过拟合强度 $\text{Overfit} = (1 - q_{\text{true}})(m-1)/(n-1)$ 是 (m, n, q_{true}) 的解析函数，其方向性可直接读出并经仿真确认： Overfit 随分区数 m 单调增、随样本量 n 单调减、随真实效应 q_{true} 单调减；功效 $1 - \beta$ 随 q_{true} 与 n 单调增；选择性偏差 $\mathbb{E}[\max_{\theta} q]$ 随搜索空间 $|\Theta|$ 按 $\ln |\Theta|$ 增长。这些方向与定理预测完全一致，未出现符号反转。

为比较各参数的相对影响，本文对基准情形施加 $\pm 20\%$ 扰动并排序。结果表明，对过拟合强度的敏感性排序为 $m > n > q_{\text{true}}$ ：分区数 m 的扰动引起的过拟合变化最大，样本量 n 次之，真实效应 q_{true} 影响相对最小。这一排序具有明确的实践含义——控制过拟合最有效的杠杆是审慎选择分区数，其次是增加样本量，而非寄望于信号本身的强弱。显著性水平 α 仅影响拒绝阈值进而调节两类错误的相对大小，不改变偏差的解析结构。

9.3 鲁棒性检验

本文从三个角度检验结论的鲁棒性。其一，对 Y 施加噪声重抽样后， q_{adj} 与 q^{CV} 的估计保持稳定，变异系数小于 5%。其二，在非正态 Y 下，置换检验与 F 检验给出一致的显著性判定，说明结论不依赖正态假设。其三，对仅含 5 个样本点的小样本区采用 Welch 自由度，避免了方差低估导致的检验失真。三项检验共同表明，本文的偏差刻画与消减方法在数据偏离理想假设时仍然可靠，所得规律具有普适性而不依赖特定数据集。

十、模型评价与推广

10.1 模型优点

本文模型的首要优点在于理论的统一性与普适性。通过证明 $q = \eta^2 = R^2$ 的等价关系，本文将地理探测器的过拟合问题纳入经典方差分析与回归诊断的成熟框架，使所有结论以 (m, n, q_{true}) 的解析式给出，天然独立于具体数据集，恰好满足题目“得出不依赖具体数据集的规律性结论”的硬要求。其次，模型层层递进且内核一致：从定理一的单调性、定理二的零期望，到定理三的偏差分解，再到 F 检验框架与选择性推断，每一步都建立在前一步之上，逻辑链条完整。第三，每个定理均经蒙特卡洛仿真验证，理论与数值高度吻合，并通过置换检验提供了不依赖正态假设的稳健备选。

10.2 模型局限

本文模型亦存在若干局限。其一，定理二与定理三的期望式基于比值的一阶近似，当 m/n 较大（如超过 0.3）时近似精度下降，导致 q_{adj} 过度校正；本文已显式声明这一边界并以置换检验或交叉验证替代，但尚未给出 m/n 较大时的闭式校正。其二，正态等方差假设在严重偏态或异方差数据上可能失效，虽有置换检验兜底，但功效分析仍以非中心 F 为基础。其三，选择性偏差的极值统计建模假设候选弱相关，对强相关搜索空间给出的是保守上界而非精确值。

10.3 模型推广

本文方法可向更广阔的场景推广。在空间统计领域， $q = R^2$ 的等价关系可直接迁移至地理加权回归、空间分层抽样等同样以方差分解为基础的方法，用以诊断其分区数或带宽选择中的过拟合。在一般机器学习场景中，本文“以零假设期望锚定虚高、以自由度惩罚校正偏差、以交叉验证与置换检验消减选择偏差”的三段式策略，适用于任何涉及模型复杂度选择与超参数搜索的问题，例如决策树的分裂数选择、聚类的簇数确定等。此外，调整 q 与置换校正可封装为地理探测器软件的标准输出项，使实践者在报告 q 值的同时给出其偏差校正版本与显著性 p 值，从制度上减轻过拟合误读。这一推广方向与近年来地理探测器在土壤碳制图^[12]、社会经济因子分析^[13] 等领域的广泛应用相契合，具有现实意义。

参考文献

- [1] Box G E P. Science and Statistics[J]. Journal of the American Statistical Association, 1976, 71(356): 791–799.

- [2] Wang J F, Li X H, Christakos G, et al. Geographical Detectors-Based Health Risk Assessment and its Application in the Neural Tube Defects Study of the Heshun Region, China[J]. International Journal of Geographical Information Science, 2010, 24(1): 107–127.
- [3] 王劲峰, 徐成东. 地理探测器: 原理与展望 [J]. 地理学报, 2017, 72(1): 116–134.
- [4] Song Y, Wang J, Ge Y, et al. An Optimal Parameters-Based Geographical Detector Model Enhances Geographic Characteristics of Explanatory Variables for Spatial Heterogeneity Analysis: Cases with Different Types of Spatial Data[J]. GIScience & Remote Sensing, 2020, 57(5): 593–610.
- [5] Shieh G. Improved Shrinkage Estimation of Squared Multiple Correlation Coefficient and Squared Cross-Validity Coefficient[J]. Organizational Research Methods, 2008, 11(2): 387–407.
- [6] Stone M. Cross-Validatory Choice and Assessment of Statistical Predictions[J]. Journal of the Royal Statistical Society: Series B, 1974, 36(2): 111–133.
- [7] Benjamini Y, Hochberg Y. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing[J]. Journal of the Royal Statistical Society: Series B, 1995, 57(1): 289–300.
- [8] Hoenig J M, Heisey D M. The Abuse of Power: The Pervasive Fallacy of Power Calculations for Data Analysis[J]. The American Statistician, 2001, 55(1): 19–24.
- [9] Efron B. Estimating the Error Rate of a Prediction Rule: Improvement on Cross-Validation[J]. Journal of the American Statistical Association, 1983, 78(382): 316–331.
- [10] Varma S, Simon R. Bias in Error Estimation When Using Cross-Validation for Model Selection[J]. BMC Bioinformatics, 2006, 7(1): 91.
- [11] Berk R, Brown L, Buja A, et al. Valid Post-Selection Inference[J]. The Annals of Statistics, 2013, 41(2): 802–837.
- [12] Liu Y, Chen Y, Wu Z, et al. Geographical Detector-Based Stratified Regression Kriging Strategy for Mapping Soil Organic Carbon with High Spatial Heterogeneity[J]. CATENA, 2021, 196: 104953.
- [13] Polykretis C, Alexakis D D. Spatial Stratified Heterogeneity of Fertility and Its Association with Socio-Economic Determinants Using Geographical Detector: The Case Study of Crete Island, Greece[J]. Applied Geography, 2021, 127: 102384.

附录 A 核心代码

本附录给出问题求解的完整 Python 代码。完整运行入口为 `main.py`，依次调用各子问题模块，随机种子固定为 `seed=42`，蒙特卡洛与置换重采样次数 $B = 2000$ ，交叉验证折数 $K = 5$ 。为节省篇幅，代码以小号字体双栏排版。

1.1 核心工具函数 `utils.py`

```
1  #-*- coding: utf-8 -*-
2  # _mh_autobootstrap_syspath_
3  import os as _mh_os, sys as _mh_sys
4  _mh_here = _mh_os.path.dirname(_mh_os.path.abspath(__file__))
5  if _mh_here and _mh_here not in _mh_sys.path:
6      _mh_sys.path.insert(0, _mh_here)
7
8  """公共工具：地理探测器 q 统计量及相关统计量计算。
9  核心等价关系： $q = 1 - WSS/TSS = SSB/TSS = \eta^2 = R^2$ （哑变量回归）。
10  所有方差用总体方差（分母  $n_i$ ），与题目式(2)/(3)定义一致。
11  """
12  import os, sys
13  _HERE = os.path.dirname(os.path.abspath(__file__))
14  if _HERE not in sys.path:
15      sys.path.insert(0, _HERE)
16
17  import numpy as np
18  import pandas as pd
19  import openpyxl
20  from scipy import stats
21
22  SEED = 42
23
24  #
25
26  # 数据读取
27  #
28  def load_xlsm(path, sheet='Input Data'):
29      """读取 GeoDetector .xlsm 的 Input Data 表为 DataFrame。"""
30      wb = openpyxl.load_workbook(path, data_only=True, keep_vba=False)
31      if sheet not in wb.sheetnames:
32          sheet = wb.sheetnames[0]
33      ws = wb[sheet]
34      rows = list(ws.iter_rows(values_only=True))
35      df = pd.DataFrame(rows[1:], columns=[str(c) for c in rows[0]])
36      df = df.dropna(how='all').reset_index(drop=True)
37      for c in df.columns:
38          df[c] = pd.to_numeric(df[c], errors='coerce')
39      return df
40
41
42  def data_dir():
43      return os.path.join(_HERE, '..', 'user_data', '_xlsm')
44
45
46  def load_disease():
47      return load_xlsm(os.path.join(data_dir(),
48                                  'GeoDetector_2018_Example(Disease
49                                  Dataset)_test.xlsm'))
50
51
52  def load_toy():
53      return load_xlsm(os.path.join(data_dir(),
54                                  'GeoDetector_2015_Example(Toy Dataset)D3.xlsm'))
55
56  #
57  # 核心统计量
58
59  def q_stat(y, groups):
60      """因子探测 q： $q = 1 - WSS/TSS$ （总体方差）。
61      y：数值数组；groups：分区标签数组（同长度）。"""
62      y = np.asarray(y, dtype=float)
63      groups = np.asarray(groups)
64      n = len(y)
65      if n < 2:
66          return 0.0
67      var_total = y.var() # 总体方差，lof=
68      tss = n * var_total
69      if tss <= 0:
70          return 0.0
71      wss = 0.0
72      for g in np.unique(groups):
73          yi = y[groups == g]
74          ni = len(yi)
75          if ni >= 1:
76              wss += ni * yi.var() # ni * 总体方差_i = 区内平方和
77      q = 1.0 - wss / tss
78      # 数值边界保护（理论上 q [0,1]，浮点误差可能略越界）
79      return float(np.clip(q, 0.0, 1.0))
80
81
82  def ss_decomp(y, groups):
83      """返回 (TSS, WSS, SSB)，用于守恒检验 TSS=WSS+SSB。"""
84      y = np.asarray(y, dtype=float)
85      groups = np.asarray(groups)
86      n = len(y)
87      ybar = y.mean()
88      tss = float(np.sum((y - ybar) ** 2))
89      wss = 0.0
90      ssb = 0.0
91      for g in np.unique(groups):
92          yi = y[groups == g]
93          ni = len(yi)
94          wss += float(np.sum((yi - yi.mean()) ** 2))
95          ssb += ni * (yi.mean() - ybar) ** 2
96      return tss, wss, ssb
97
98
99  def q_adj(q, n, m):
100      """调整 q（仿调整  $R^2$ ，自由度惩罚）： $q_{adj} = 1 - (1-q)(n-1)/(n-m)$ 。"""
101      if n - m <= 0:
102          return float('nan')
103      return float(1.0 - (1.0 - q) * (n - 1) / (n - m))
104
105
106  def factor_F(q, n, m):
107      """因子探测器 F 统计量： $F = [q/(m-1)] / [(1-q)/(n-m)] - F(m-1, n-m)$ 。"""
108      if m <= 1 or n - m <= 0:
109          return float('nan')
110      q = min(max(q, 0.0), 1.0 - 1e-12)
111      return float((q / (m - 1)) / ((1 - q) / (n - m)))
112
113
114  def factor_pvalue(q, n, m):
115      """因子探测器 F 检验 p 值（中心 F 零分布）。"""
116      F = factor_F(q, n, m)
117      if not np.isfinite(F):
```

```

118     return float('nan')
119     return float(stats.f.sf(F, m - 1, n - m))
120
121
122 def power_ncf(q_true, n, m, alpha=0.05):
123     """非中心 F 检验功效:  $1 - \beta = P(F(df_1, df_2; \lambda) > F_{crit})$ 。
124      $\lambda = n * q\_true / (1 - q\_true)$ ,  $q\_true$  为功效=alpha (用中心 F,
125     避免 scipy.stats.ncf 在  $\lambda=0$  处的数值缺陷)。"""
126     df1, df2 = m - 1, n - m
127     if df1 <= 0 or df2 <= 0:
128         return float('nan')
129     q_true = min(max(q_true, 0.0), 1 - 1e-9)
130     Fc = stats.f.ppf(1 - alpha, df1, df2)
131     lam = n * q_true / (1 - q_true) if q_true < 1 else 1e9
132     if lam < 1e-8:
133         return float(stats.f.sf(Fc, df1, df2)) # 中心 F, 理论上等于 alpha
134     return float(stats.ncf.sf(Fc, df1, df2, lam))
135
136 # -----
137 # 蒙特卡洛 / 置换
138 # -----
139
140 def random_partition(n, m, rng):
141     """把 n 个样本随机分到 m 区 (各区至少不强), 返回标签数组。"""
142     return rng.integers(0, m, size=n)
143
144
145 def _balanced_groups(n, m):
146     """生成平衡分区标签 (各区样本数尽量相等), 用于含信号仿真。"""
147     base = np.repeat(np.arange(m), n // m)
148     rem = n - len(base)
149     if rem > 0:
150         base = np.concatenate([base, np.arange(rem)])
151     return base
152
153
154 def _fixed_signal_means(m, q_true):
155     """构造确定性组均值  $\mu_i$ : 等权方差恰为  $q\_true$ , 残差方差  $1 - q\_true$ 。
156     使非中心参数  $= n * q\_true / (1 - q\_true)$ , 从而
157      $E[\hat{q}] = q\_true + (1 - q\_true)(m - 1) / (n - 1)$ 。"""
158     base = np.linspace(-1, 1, m)
159     base = base - base.mean()
160     var_b = base.var() # 等权总体方差 (ddof=0)
161     if var_b <= 0:
162         return np.zeros(m)
163     return base * np.sqrt(q_true / var_b)
164
165
166 def monte_carlo_q(n, m, reps=2000, signal='noise', q_true=0.0, seed=SEED):
167     """蒙特卡洛  $E[q]$ 。
168     signal: 'noise' 纯噪声 (H0, 随机分区);
169     'signal' 含真实信号: 确定性组均值 (等权方差=q_true) +
170     残差方差=(1-q_true),
171     平衡分区, 使理论  $E[\hat{q}] = q\_true + (1 - q\_true)(m - 1) / (n - 1)$ 
172     成立。
173     返回 (mean_q, q_array)。"""
174     rng = np.random.default_rng(seed)
175     qs = np.empty(reps)
176     if signal == 'signal':
177         g = _balanced_groups(n, m)
178         means = _fixed_signal_means(m, q_true)
179         sigma_e = np.sqrt(max(1.0 - q_true, 1e-9))
180         mu_vec = means[g]
181         for r in range(reps):
182             if signal == 'noise':
183                 y = rng.standard_normal(n)
184             else:
185                 y = mu_vec + rng.normal(0, sigma_e, size=n)
186
187         qs[r] = q_stat(y, g)
188     return float(qs.mean()), qs
189
190
191 def welch_t_pair(a, b):
192     """两组 Welch t 检验, 返回 (t, p_two_sided, df)。"""
193     a = np.asarray(a, dtype=float); b = np.asarray(b, dtype=float)
194     na, nb = len(a), len(b)
195     if na < 2 or nb < 2:
196         return float('nan'), float('nan'), float('nan')
197     va, vb = a.var(ddof=1), b.var(ddof=1)
198     se = np.sqrt(va / na + vb / nb)
199     if se == 0:
200         return float('nan'), float('nan'), float('nan')
201     t = (a.mean() - b.mean()) / se
202     df = (va / na) ** 2 / (na - 1) + (vb / nb) ** 2 / (nb - 1)
203     p = 2 * stats.t.sf(abs(t), df)
204     return float(t), float(p), float(df)
205
206
207 def bh_fdr(pvals, alpha=0.05):
208     """Benjamini-Hochberg FDR: 返回 (reject_bool_array, n_reject)。"""
209     p = np.asarray(pvals, dtype=float)
210     valid = ~np.isnan(p)
211     pv = p[valid]
212     c = len(pv)
213     if c == 0:
214         return np.zeros_like(p, dtype=bool), 0
215     order = np.argsort(pv)
216     thresh = (np.arange(1, c + 1) / c) * alpha
217     passed = pv[order] <= thresh
218     kmax = np.where(passed)[0]
219     reject_sorted = np.zeros(c, dtype=bool)
220     if len(kmax) > 0:
221         k = kmax.max()
222         reject_sorted[order[:k + 1]] = True
223     out = np.zeros_like(p, dtype=bool)
224     out[np.where(valid)[0]] = reject_sorted
225     return out, int(reject_sorted.sum())
226
227
228 def bonferroni(pvals, alpha=0.05):
229     """Bonferroni: 返回 (reject_bool_array, n_reject)。"""
230     p = np.asarray(pvals, dtype=float)
231     c = np.sum(~np.isnan(p))
232     reject = p <= (alpha / max(c, 1))
233     reject = reject & ~np.isnan(p)
234     return reject, int(reject.sum())
235
236
237 def interaction_groups(x1, x2):
238     """交互探测器: 叠置 X1 X2 形成联合分区标签。"""
239     x1 = np.asarray(x1); x2 = np.asarray(x2)
240     return np.array([f"{a}_{b}" for a, b in zip(x1, x2)])
241
242
243 def permutation_null(y, groups, B=2000, seed=SEED):
244     """置换检验: 固定分区结构, 随机置换 Y 标签, 得 q 的经验零分布。
245     返回 (q_obs, null_array, p_value)。"""
246     rng = np.random.default_rng(seed)
247     y = np.asarray(y, dtype=float)
248     groups = np.asarray(groups)
249     q_obs = q_stat(y, groups)
250     null = np.empty(B)
251     for b in range(B):
252         yp = rng.permutation(y)
253         null[b] = q_stat(yp, groups)
254     p = (1 + np.sum(null >= q_obs)) / (B + 1)
255     return float(q_obs), null, float(p)

```

1.2 问题一: $q-m$ 关系与零假设期望

```

1 # -*- coding: utf-8 -*-
2 # __mh_autobootstap_sys_path__
3 import os as _mh_os, sys as _mh_sys
4 _mh_here = _mh_os.path.dirname(_mh_os.path.abspath(__file__))
5 if _mh_here and _mh_here not in _mh_sys.path:
6     _mh_sys.path.insert(0, _mh_here)
7
8 """问题一: q 统计量与分区数量 m 的关系。
9 验证: (1) 嵌套细化单调性; (2) 边界值 m=1->q=0, m=n->q=1
10 (3) 零假设期望 E[q|H0]=(m-1)/(n-1); (4) noise/weak/strong 三档信号 q(m) 曲线族。
11 输出: figures/problem_1_results.json
12 """
13 import os, sys
14 _HERE = os.path.dirname(os.path.abspath(__file__))
15 if _HERE not in sys.path:
16     sys.path.insert(0, _HERE)
17
18 import json
19 import numpy as np
20 import utils as u
21
22 FIG_DIR = os.path.join(_HERE, '..', 'figures')
23 os.makedirs(FIG_DIR, exist_ok=True)
24
25
26 def verify_boundary(n=100, seed=u.SEED):
27     """边界值检验: m=1 -> q=0; m=n -> q=1。"""
28     rng = np.random.default_rng(seed)
29     y = rng.standard_normal(n)
30     q_m1 = u.q_stat(y, np.zeros(n, dtype=int)) # 全部一区
31     q_mn = u.q_stat(y, np.arange(n)) # 每点一区
32     return {'m=1': q_m1, 'm=n': q_mn}
33
34
35 def verify_monotonicity(n=120, seed=u.SEED):
36     """嵌套细化单调性: 逐步细化分区, q 必非降。"""
37     rng = np.random.default_rng(seed)
38     y = rng.standard_normal(n)
39     # 从 2 区开始, 每次把某一区再二分(细化), 记录 q
40     labels = (np.arange(n) >= n // 2).astype(int) # 初始 2 区
41     qs = [u.q_stat(y, labels)]
42     ms = [2]
43     cur_m = 2
44     for step in range(8):
45         # 选最大的区二分
46         counts = np.bincount(labels)
47         big = np.argmax(counts)
48         idx = np.where(labels == big)[0]
49         half = idx[len(idx) // 2]
50         labels[half] = cur_m # 新标签 = 细化
51         cur_m += 1
52         qs.append(u.q_stat(y, labels))
53         ms.append(len(np.unique(labels)))
54     monotone = all(qs[i + 1] >= qs[i] - 1e-9 for i in range(len(qs) - 1))
55     return {'m_sequence': ms, 'q_sequence': [float(x) for x in qs],
56           'is_monotone_nondecreasing': bool(monotone)}
57
58
59 def null_expectation(n=100, m_list=(2, 5, 10, 20, 50, 100), reps=2000,
60                     seed=u.SEED):
61     """零假设期望 E[q|H0] vs 理论 (m-1)/(n-1)。"""
62     out = {'m': [], 'E_q_empirical': [], 'theory': [], 'rel_err': []}
63     for m in m_list:
64         mean_q, _ = u.monte_carlo_q(n, m, reps=reps, signal='noise',
65                                     seed=seed)
66         theo = (m - 1) / (n - 1)
67         out['m'].append(int(m))
68         out['E_q_empirical'].append(round(mean_q, 4))
69         out['theory'].append(round(theo, 4))
70         rel = abs(mean_q - theo) / theo if theo > 0 else 0.0
71         out['rel_err'].append(round(rel, 4))
72     return out
73
74 def q_vs_m_curves(n=120, m_list=None, reps=1500, seed=u.SEED):
75
76     """noise / weak / strong 三档信号下 q(m) 曲线族 + 分位带。"""
77     if m_list is None:
78         m_list = list(range(2, 61, 2))
79     signals = {'noise': 0.0, 'weak': 0.2, 'strong': 0.6}
80     curves = {}
81     for name, qt in signals.items():
82         mean_curve, lo, hi = [], [], []
83         for m in m_list:
84             sig = 'noise' if name == 'noise' else 'signal'
85             _, qs = u.monte_carlo_q(n, m, reps=reps, signal=sig,
86                                     q_true=qt, seed=seed)
87             mean_curve.append(round(float(np.mean(qs)), 4))
88             lo.append(round(float(np.percentile(qs, 5)), 4))
89             hi.append(round(float(np.percentile(qs, 95)), 4))
90             curves[name] = {'q_true': qt, 'mean': mean_curve, 'p5': lo, 'p95': hi}
91     theory = [round((m - 1) / (n - 1), 4) for m in m_list]
92     return {'m_list': list(m_list), 'n': n, 'curves': curves,
93           'theory_null': theory}
94
95 def validate_constraints(res):
96     """问题一结果的验证。"""
97     ok = True
98     msgs = []
99     # 边界值
100     b = res['boundary']
101     if abs(b['m=1']) > 1e-6:
102         ok = False; msgs.append(f"m=1 时 q={b['m=1']} 应为 0")
103     else:
104         msgs.append("m=1 -> q=0")
105     if abs(b['m=n'] - 1.0) > 1e-6:
106         ok = False; msgs.append(f"m=n 时 q={b['m=n']} 应为 1")
107     else:
108         msgs.append("m=n -> q=1")
109     # 单调性
110     if not res['monotonicity']['is_monotone_nondecreasing']:
111         ok = False; msgs.append("细化分区 q 非单调非降")
112     else:
113         msgs.append("嵌套细化 q 单调非降")
114     # 零假设期望 (m<n 区间 rel_err<5%)
115     ne = res['null_expectation']
116     bad = [m for m, m_ in zip(ne['m'], ne['rel_err'])
117            if m <= 20 and m_ > 0.05]
118     if bad:
119         ok = False; msgs.append(f"m<n 时 E[q|H0] 偏差>5%: m={bad}")
120     else:
121         msgs.append("m<n 时 E[q|H0] (m-1)/(n-1) (rel_err<5%)")
122     # q 全部在 [0,1]
123     allq = []
124     for c in res['q_vs_m']['curves'].values():
125         allq += c['mean'] + c['p5'] + c['p95']
126     if min(allq) < -1e-9 or max(allq) > 1 + 1e-9:
127         ok = False; msgs.append("存在 q 越界 [0,1]")
128     else:
129         msgs.append("所有 q [0,1]")
130     return ok, msgs
131
132 def main():
133     print("[问题一] 计算边界值 / 单调性 / 零期望 / q(m) 曲线族 ...")
134     res = {}
135     res['boundary'] = verify_boundary()
136     print("边界值:", res['boundary'])
137     res['monotonicity'] = verify_monotonicity()
138     print("单调性 OK:", res['monotonicity']['is_monotone_nondecreasing'])
139     res['null_expectation'] = null_expectation()
140     print("零期望表:")
141     for m, e, t in zip(res['null_expectation']['m'],
142                       res['null_expectation']['E_q_empirical'],
143                       res['null_expectation']['theory']):
144         print(f"m={m:3d} E[q]={e:.4f} theory={t:.4f}")
145     res['q_vs_m'] = q_vs_m_curves()
146     print("q(m) 曲线族完成 (noise/weak/strong)")
147
148     ok, msgs = validate_constraints(res)

```

```

149 print(" 验证成功")
150 for mm in msg:
151     print("    ", mm)
152 res['validation'] = {'passed': ok, 'messages': msg}
153
154 out = os.path.join(FIG_DIR, 'problem_1_results.json')
155 with open(out, 'w', encoding='utf-8') as f:
156     json.dump(res, f, ensure_ascii=False, indent=2)
157     print(" 已保存:", out)
158     return res
159
160
161 if __name__ == '__main__':
162     main()

```

1.3 问题二：过拟合偏差分解与调整 q

```

1  #-*- coding: utf-8 -*-
2  # __mh_autobootstrap_syspath__
3  import os as _mh_os, sys as _mh_sys
4  _mh_here = _mh_os.path.dirname(_mh_os.path.abspath(__file__))
5  if _mh_here and _mh_here not in _mh_sys.path:
6      _mh_sys.path.insert(0, _mh_here)
7
8  """问题二：过拟合的发生条件与强度（理论模型）。
9  核心定理3:  $E[\hat{q}] = q_{\text{true}} + (1-q_{\text{true}})(m-1)/(n-1)$ 。
10  过拟合强度 Overfit =  $(1-q_{\text{true}})(m-1)/(n-1)$ ；调整
11       $q: q_{\text{adj}} = 1 - (1-q)(n-1)/(n-m)$ 。
12  输出: figures/problem_2_results.json
13  """
14  import os, sys
15  _HERE = os.path.dirname(os.path.abspath(__file__))
16  if _HERE not in sys.path:
17      sys.path.insert(0, _HERE)
18
19  import json
20  import numpy as np
21  import utils as u
22
23  FIG_DIR = os.path.join(_HERE, '..', 'figures')
24  os.makedirs(FIG_DIR, exist_ok=True)
25
26  def bias_decomp_verify(seed=u.SEED):
27      """定理3 验证:  $E[\hat{q}]$  理论 vs 蒙特卡洛。"""
28      cases = [
29          {'n': 120, 'm': 6, 'q_true': 0.318},
30          {'n': 100, 'm': 5, 'q_true': 0.0},
31          {'n': 150, 'm': 10, 'q_true': 0.5},
32          {'n': 200, 'm': 8, 'q_true': 0.2},
33      ]
34      rows = []
35      for c in cases:
36          n, m, qt = c['n'], c['m'], c['q_true']
37          theo = qt + (1 - qt) * (m - 1) / (n - 1)
38          sig = 'noise' if qt == 0 else 'signal'
39          emp, _ = u.monte_carlo_q(n, m, reps=2000, signal=sig, q_true=qt,
40                                  seed=seed)
41          rows.append({'n': n, 'm': m, 'q_true': qt,
42                      'E_ghat_theory': round(theo, 4),
43                      'E_ghat_empirical': round(emp, 4),
44                      'abs_err': round(abs(theo - emp), 4)})
45      return rows
46
47  def overfit_heatmap(m_max=40, n_list=None, q_true=0.0):
48      """过拟合强度 Overfit(m,n) 在 (m,n) 平面（默认纯噪声 q_true=0）。"""
49      if n_list is None:
50          n_list = list(range(20, 401, 20))
51      m_list = list(range(2, m_max + 1))
52      Z = [] # Z[i][j] = Overfit at n_list[i], m_list[j]
53      for n in n_list:
54          row = []
55          for m in m_list:
56              if m < n:
57                  ov = (1 - q_true) * (m - 1) / (n - 1)
58              else:
59                  ov = float('nan')
60              row.append(round(ov, 4) if np.isfinite(ov) else None)
61
62          Z.append(row)
63      return {'m_list': m_list, 'n_list': n_list, 'q_true': q_true, 'Z': Z}
64
65  def empirical_q_adj():
66      """疾病数据 type/region/level 的 q, q_adj, 差值（过拟合校正量）。"""
67      dis = u.load_disease()
68      y = dis['in_disease'].values
69      n = len(dis)
70      rows = []
71      for v in ['type', 'region', 'level']:
72          g = dis[v].values
73          m = int(np.unique(g).size)
74          q = u.q_stat(y, g)
75          qa = u.q_adj(q, n, m)
76          rows.append({'variable': v, 'm': m, 'n': n,
77                      'q': round(q, 4), 'q_adj': round(qa, 4),
78                      'overfit_correction': round((q - qa), 4)})
79      return rows
80
81
82  def qadj_null_check(n=100, m_list=(5, 20), reps=2000, seed=u.SEED):
83      """验证 H0 下  $E[q_{\text{adj}}] \approx 0$  ( $m \ll n$ )；记录 m/n 大时的过度校正边界。"""
84      out = []
85      for m in m_list:
86          _, qs = u.monte_carlo_q(n, m, reps=reps, signal='noise', seed=seed)
87          qadjs = np.array([u.q_adj(q, n, m) for q in qs])
88          out.append({'n': n, 'm': m, 'E_q': round(float(qs.mean()), 4),
89                     'E_q_adj': round(float(qadjs.mean()), 4)})
90      # 额外记录 m/n 大的边界 (m=50)
91      _, qs50 = u.monte_carlo_q(n, 50, reps=reps, signal='noise', seed=seed)
92      qadj50 = np.array([u.q_adj(q, n, 50) for q in qs50])
93      out.append({'n': n, 'm': 50, 'E_q': round(float(qs50.mean()), 4),
94                 'E_q_adj': round(float(qadj50.mean()), 4),
95                 'note': 'm/n=0.5 过度校正边界, q_adj 系统性偏差, 应采用置换/CV'})
96      return out
97
98
99  def discretization_amplification(n=120, m=5, n_thresholds_list=(5, 20, 60),
100                                  reps=1000, seed=u.SEED):
101      """离散化多重比较放大：纯噪声下，在 N 个候选切分点里挑选使 q 最大的离散化，
102      对比‘先验固定切分’与‘数据驱动挑选’的  $E[q]$ 。验证数据驱动偏差被放大。"""
103      rng = np.random.default_rng(seed)
104      out = []
105      for ncand in n_thresholds_list:
106          q_fixed = np.empty(reps)
107          q_datadriven = np.empty(reps)
108          for r in range(reps):
109              x = rng.standard_normal(n) # 连续型 X (与 Y 独立 -> H0)
110              y = rng.standard_normal(n)
111              # 先验固定切分：用 X 的等距分位点切成 m 段（不看 Y）
112              qs_fixed = np.quantile(x, np.linspace(0, 1, m + 1)[1:-1])
113              g_fixed = np.digitize(x, qs_fixed)
114              q_fixed[r] = u.q_stat(y, g_fixed)
115              # 数据驱动：在 ncand 个候选切分方案中挑选 q 最大者
116              best = -1.0
117              for _ in range(ncand):
118                  cuts = np.sort(rng.choice(x, size=m - 1, replace=False))
119                  g = np.digitize(x, cuts)
120                  qq = u.q_stat(y, g)
121                  if qq > best:

```

```

122     best = qq
123     q_data driven[r] = best
124     out.append({'candidate': int(ncand),
125                'E_q_fixed': round(float(q_fixed.mean()), 4),
126                'E_q_data driven': round(float(q_data driven.mean()), 4),
127                'amplification_ratio': round(
128                    float(q_data driven.mean() / max(q_fixed.mean(),
129                1e-6)), 2)})
129     return out
130
131 def validate_constraints(res):
132     ok = True
133     msgs = []
134     # 定理3 偏差
135     bad = [r for r in res['bias_decomp'] if r['abs_err'] > 0.02]
136     if bad:
137         ok = False; msgs.append(f" 定理3 偏差>0.02: {[r['n'], r['m']] for r in bad}")
138     else:
139         msgs.append(" 定理3 E[q_hat] 理论=蒙特卡洛 (abs_err<0.02)")
140     # q_adj <= 1
141     qadj_vals = [r['q_adj'] for r in res['empirical']] + \
142                 [r['E_q_adj'] for r in res['qadj_null']]
143     if max(qadj_vals) > 1 + 1e-9:
144         ok = False; msgs.append(" q_adj>1 不可能")
145     else:
146         msgs.append(" 所有 q_adj 1")
147     # H0 下 m<n 时 E[q_adj] 0
148     for r in res['qadj_null']:
149         if r['m'] <= 20 and abs(r['E_q_adj']) > 0.02:
150             ok = False; msgs.append(f" m={r['m']} 时
151             E[q_adj]={r['E_q_adj']} 偏离0")
152     if ok:
153         msgs.append(" H0 下 m<n 时 E[q_adj] 0")
154     # 离散化放大: 数据驱动 E[q] > 固定切分 E[q]
155     amp_ok = all(r['E_q_data driven'] >= r['E_q_fixed'] for r in
156     res['discretization'])
157     if not amp_ok:
158         ok = False; msgs.append(" 数据驱动离散化未放大偏差")
159     else:
160         msgs.append(" 数据驱动离散化偏差 > 固定切分 (多重比较放大已验证)")
161     # 方向性: m↑ Overfit↑ (用 heatmap 同一 n 检查)
162     hm = res['overfit_heatmap']
163     row0 = [v for v in hm['Z'] if len(hm['n_list']) // 2 if v is not None]
164     if not all(row0[i+1] >= row0[i] - 1e-9 for i in range(len(row0) -
165     1)):
166         ok = False; msgs.append(" Overfit 关于 m 非单调增")
167     else:
168         msgs.append(" Overfit 关于 m 单调增 (方向正确)")
169     return ok, msgs
170
171 def main():
172     print("[问题二] 偏差分解 / 调整q / 过拟合响应面 / 离散化放大 ...")
173     res = {}
174     res['bias_decomp'] = bias_decomp_verify()
175     print(" 定理3 验证:")
176     for r in res['bias_decomp']:
177         print(f" n={r['n']} m={r['m']} q_true={r['q_true']}: "
178               f"理论={r['E_qhat_theory']} 实测={r['E_qhat_empirical']}")
179     res['overfit_heatmap'] = overfit_heatmap()
180     res['empirical'] = empirical_q_qadj()
181     print(" 实证 q/q_adj:")
182     for r in res['empirical']:
183         print(f" {r['variable']}: {r['m']}={r['m']} q={r['q']}
184               q_adj={r['q_adj']}")
185     # 离散化放大=overfit_correction
186     res['qadj_null'] = qadj_null_check()
187     print(" q_adj 零期望:")
188     for r in res['qadj_null']:
189         print(f" m={r['m']} q={r['q']} E[q_adj]={r['E_q_adj']}")
190     res['discretization'] = discretization_amplification()
191     print(" 离散化多重比较放大")
192     for r in res['discretization']:
193         print(f" 候选数={r['n_candidate']} 固定={r['E_q_fixed']} "
194               f"数据驱动={r['E_q_data driven']}
195               放大={r['amplification_ratio']}x")
196
197     ok, msgs = validate_constraints(res)
198     print("[约束验证]")
199     for mm in msgs:
200         print(" ", mm)
201     res['validation'] = {'passed': ok, 'messages': msgs}
202
203     out = os.path.join(FIG_DIR, 'problem_2_results.json')
204     with open(out, 'w', encoding='utf-8') as f:
205         json.dump(res, f, ensure_ascii=False, indent=2)
206     print(" 已保存:", out)
207     return res
208
209 if __name__ == '__main__':
210     main()

```

1.4 问题三：四探测器假设检验框架

```

1  # -*- coding: utf-8 -*-
2  # __mh_autobootstrap_syspath__
3  import os as _mh_os, sys as _mh_sys
4  _mh_here = _mh_os.path.dirname(_mh_os.path.abspath(__file__))
5  if _mh_here and _mh_here not in _mh_sys.path:
6      _mh_sys.path.insert(0, _mh_here)
7
8  """问题三：四探测器统一假设检验框架 + 两类错误。
9  因子(F检验) / 风险(成对Welch t + FDR/Bonferroni) / 交互(叠置 q_adj) /
10  生态(F比)。
11  功效函数(非中心F) + 两类错误权衡 + 置换检验经验零分布。
12  输出: figures/problem_3_results.json
13  """
14  import os, sys
15  _HERE = os.path.dirname(os.path.abspath(__file__))
16  if _HERE not in sys.path:
17      sys.path.insert(0, _HERE)
18
19  import json
20  import itertools
21  import numpy as np
22  from scipy import stats
23
24  import utils as u
25  FIG_DIR = os.path.join(_HERE, '..', 'figures')
26  os.makedirs(FIG_DIR, exist_ok=True)
27
28  def factor_detector(dis):
29      """因子探测器: 对 type/region/level 计算 F, p, F_crit, 是否显著。"""
30      y = dis['incidence'].values
31      n = len(dis)
32      rows = []
33      for v in ['type', 'region', 'level']:
34          g = dis[v].values
35          m = int(np.unique(g).size)
36          q = u.q_stat(y, g)
37          F = u.factor_F(q, n, m)
38          p = u.factor_pvalue(q, n, m)
39          Fc = float(stats.f.ppf(0.95, m - 1, n - m))
40          rows.append({'variable': v, 'm': m, 'n': n, 'q': round(q, 4),
41                      'q_adj': round(u.q_adj(q, n, m), 4),
42                      'F': round(F, 2), 'p_value': p,
43                      'F_crit_0.05': round(Fc, 3),

```



```

44         'significant': bool(F > Fc))
45     return rows
46
47
48 def risk_detector(dis, var='region', alpha=0.05):
49     """风险探测器: region 的所有成对 welch t 检验 + 多重比较校正。"""
50     y = dis['incidence'].values
51     g = dis[var].values
52     cats = sorted(np.unique(g).tolist())
53     m = len(cats)
54     pairs = list(itertools.combinations(cats, 2))
55     c = len(pairs) # C(m,2)
56     rows = []
57     pvals = []
58     for a, b in pairs:
59         ya = y[g == a]; yb = y[g == b]
60         t, p, df = u.welch_t_pair(ya, yb)
61         diff = float(ya.mean() - yb.mean())
62         se = abs(diff / t) if (t and np.isfinite(t) and t != 0) else
float('nan')
63         rows.append({'pair': f"{a}-{b}", 'mean_diff': round(diff, 4),
64                     't': round(t, 3) if np.isfinite(t) else None,
65                     'p_value': p, 'se': round(se, 4) if np.isfinite(se)
66                     else None,
67                     'df': round(df, 1) if np.isfinite(df) else None})
68         pvals.append(p)
69     pvals = np.array(pvals)
70     raw_reject = int(np.isnansum(pvals < alpha))
71     bonf_mask, bonf_n = u.bonferroni(pvals, alpha)
72     bh_mask, bh_n = u.bh_fdr(pvals, alpha)
73     fwer_uncorrected = float(1 - (1 - alpha) ** c)
74     for i, r in enumerate(rows):
75         r['sig_raw'] = bool(np.isfinite(pvals[i]) and pvals[i] < alpha)
76         r['sig_bonferroni'] = bool(bonf_mask[i])
77         r['sig_bh_fdr'] = bool(bh_mask[i])
78     return {'variable': var, 'm': m, 'n_pairs': c,
79             'fwer_uncorrected_approx': round(fwer_uncorrected, 4),
80             'n_sig_raw': raw_reject, 'n_sig_bonferroni': bonf_n,
81             'n_sig_bh_fdr': bh_n, 'pairs': rows}
82
83 def interaction_detector(dis):
84     """交互探测器: 所有变量对叠置, 比较 q_adj(X1,X2) 与
85         max(q_adj(X1),q_adj(X2))。
86         用 q_adj 而非原始 q 防止 m 暴涨导致虚假增强。"""
87     y = dis['incidence'].values
88     n = len(dis)
89     varlist = ['type', 'region', 'level']
90
91     def qadj_of(g):
92         m = int(np.unique(g).size)
93         return u.q_adj(u.q_stat(y, g), n, m), m
94
95     single = {}
96     for v in varlist:
97         qa, m = qadj_of(dis[v].values)
98         single[v] = {'q': round(u.q_stat(y, dis[v].values), 4),
99                     'q_adj': round(qa, 4), 'm': m}
100
101     pairs = []
102     matrix_vars = varlist
103     qadj_matrix = [[None] * len(matrix_vars) for _ in matrix_vars]
104     for i, v1 in enumerate(matrix_vars):
105         qadj_matrix[i][i] = single[v1]['q_adj']
106     for v1, v2 in itertools.combinations(varlist, 2):
107         g12 = u.interaction_groups(dis[v1].values, dis[v2].values)
108         q12 = u.q_stat(y, g12)
109         qa12, m12 = qadj_of(g12)
110         qa1 = single[v1]['q_adj']; qa2 = single[v2]['q_adj']
111         # 增强/减弱五分类 (基于 q_adj 公允比较)
112         mn, mx = min(qa1, qa2), max(qa1, qa2)
113         ssum = qa1 + qa2
114         if qa12 < mn:
115             cls = '非线性减弱'
116         elif qa12 < mx:
117             cls = '单因子非线性减弱'
118         elif qa12 < ssum:
119             cls = '双因子增强'
120         elif abs(qa12 - ssum) < 1e-6:
121             cls = '独立'
122         else:
123             cls = '非线性增强'
124         i1, i2 = matrix_vars.index(v1), matrix_vars.index(v2)
125         qadj_matrix[i1][i2] = round(qa12, 4)
126         qadj_matrix[i2][i1] = round(qa12, 4)
127         pairs.append({'pair': f"{v1}{v2}", 'm12': m12,
128                     'q_raw': round(q12, 4),
129                     'q_adj_joint': round(qa12, 4),
130                     'q_adj_x1': round(qa1, 4), 'q_adj_x2': round(qa2, 4),
131                     'classification': cls})
132     return {'single': single, 'matrix_vars': matrix_vars,
133             'q_adj_matrix': qadj_matrix, 'pairs': pairs}
134
135 def ecological_detector(dis):
136     """生态探测器: F 比 = [WSS_X1/(n-1)] / [WSS_X2/(n-2)] (题目式6)。"""
137     y = dis['incidence'].values
138     n = len(dis)
139     varlist = ['type', 'region', 'level']
140     wss = {}
141     mm = {}
142     for v in varlist:
143         _, w, _ = u.ss_decomp(dis[v].values)
144         wss[v] = w
145         mm[v] = int(np.unique(dis[v].values).size)
146     rows = []
147     for v1, v2 in itertools.combinations(varlist, 2):
148         df1, df2 = n - mm[v1], n - mm[v2]
149         F = (wss[v1] / df1) / (wss[v2] / df2)
150         # 双侧 F 检验
151         p = 2 * min(stats.f.sf(F, df1, df2), stats.f.cdf(F, df1, df2))
152         rows.append({'pair': f"{v1} vs {v2}", 'F_ratio': round(F, 4),
153                     'df1': df1, 'df2': df2, 'p_value': round(float(p), 4),
154                     'significant_diff': bool(p < 0.05)})
155     return rows
156
157
158 def power_curves(n_list=(50, 100, 200), m_list=(3, 6, 12),
159                 q_true_grid=None, alpha=0.05):
160     """功效函数 1- vs q_true (不同 m,n)。"""
161     if q_true_grid is None:
162         q_true_grid = list(np.round(np.linspace(0.0, 0.6, 25), 3))
163     curves = []
164     for n in n_list:
165         for m in m_list:
166             if m >= n:
167                 continue
168             power = [round(u.power_ncf(qt, n, m, alpha), 4) for qt in
169                     q_true_grid]
170             curves.append({'n': n, 'm': m, 'power': power})
171     return {'q_true_grid': [float(x) for x in q_true_grid], 'alpha': alpha,
172             'curves': curves}
173
174 def type1_type2_tradeoff(n=120, q_true=0.15, m_list=None, alpha=0.05,
175                          reps=2000, seed=u.SEED):
176     """两类错误随 m 的权衡。
177     : 名义=alpha; 当数据驱动离散化时实际膨胀 (用纯噪声下 max 离散化估计)。
178     : 1 - power_ncf(q_true,n,m)。"""
179     if m_list is None:
180         m_list = list(range(2, 41, 2))
181     rng = np.random.default_rng(seed)
182     out = {'m_list': list(m_list), 'n': n, 'q_true': q_true,
183           'alpha_nominal': alpha, 'beta': [], 'power': [],
184           'alpha_actual_datadriven': []}
185     for m in m_list:
186         beta = 1 - u.power_ncf(q_true, n, m, alpha)
187         out['beta'].append(round(float(beta), 4))
188         out['power'].append(round(float(1 - beta), 4))
189         # 实际: 纯噪声下做数据驱动离散化(20候选)后 F 检验的拒绝率
190         rej = 0
191         R = 400

```



```

192 for _ in range(5):
193     x = rng.standard_normal(n); yv = rng.standard_normal(n)
194     best_q = 1
195     for _ in range(20):
196         cuts = np.sort(rng.choice(x, size=m - 1, replace=False))
197         if m > 1 else np.array([])
198         g = np.digitize(x, cuts)
199         qq = u.q_stat(yv, g)
200         if qq > best_q:
201             best_q = qq
202         p = u.factor_pvalue(best_q, n, m)
203         if np.isfinite(p) and p < alpha:
204             rej += 1
205     out['alpha_actual_datadriven'].append(round(rej / 4))
206 # 最优 m* (最小化 _actual + )
207 total = [a + b for a, b in zip(out['alpha_actual_datadriven'],
208                                out['beta'])]
209 out['m_star'] = int(m_list[int(np.argmin(total))])
210 return out
211
212 def permutation_demo(dis, m_demo=(3, 6, 12), n=100, B=2000, seed=u.SEED):
213     """置换检验经验分布: 纯噪声下不同 m 的 q 经验分布 (不依赖正态)"""
214     rng = np.random.default_rng(seed)
215     out = {'m_values': list(m_demo), 'n': n, 'B': B, 'null_samples': {}}
216     for m in m_demo:
217         y = rng.standard_normal(n)
218         g = u._balanced_groups(n, m)
219         _, null, _ = u.permutation_null(y, g, B=B, seed=seed)
220         out['null_samples'][str(m)] = {
221             'mean': round(float(null.mean()), 4),
222             'theory_mean': round((m - 1) / (n - 1), 4),
223             'p5': round(float(np.percentile(null, 5)), 4),
224             'p95': round(float(np.percentile(null, 95)), 4),
225             'samples': [round(float(x), 4) for x in null[:500]]}
226     return out
227
228 def validate_constraints(res):
229     ok = True; msgs = []
230     # 因子: F>0, p [0,1]
231     for r in res['factor']:
232         if r['F'] <= 0 or not (0 <= r['p_value'] <= 1):
233             ok = False; msgs.append(f" 因子 {r['variable']} F/p 异常")
234     msgs.append(" 因子探测器 F>0, p [0,1] 如果 ok else None
235     # 风险: 校正后显著数 <= 原始
236     rk = res['risk']
237     if not (rk['n_sig_bonferroni'] <= rk['n_sig_raw'] and
238            rk['n_sig_bh_fdr'] <= rk['n_sig_raw']):
239         ok = False; msgs.append(" 多重比较校正后显著数未减少")
240     else:
241         msgs.append(f" 多重比较校正:
242             原始{rk['n_sig_raw']}→Bonf{rk['n_sig_bonferroni']}/BH{rk['n_sig_bh_fdr']}
243             # FWER 未校正应很大
244             if rk['fwer_uncorrected_approx'] < 0.5:
245                 msgs.append(f" FWER未校正={rk['fwer_uncorrected_approx']}
246                     (m={rk['m']})")
247             else:
248                 msgs.append(f" FWER未校正 {rk['fwer_uncorrected_approx']} (膨胀,
249                     需校正)")
250             # 功效 [0,1] 且对 q_true 单调非降
251             for cv in res['power']['curves']:
252                 p = cv['power']
253                 if min(p) < -1e-9 or max(p) > 1 + 1e-9:
254                     ok = False; msgs.append(" 功效越界 [0,1]")
255                     if not all(p[i+1] >= p[i] - 1e-6 for i in range(len(p)-1)):
256                         ok = False; msgs.append(f" 功效对 q_true 非单调
257                             (n={cv['n']},m={cv['m']})")
258             if ok:
259                 msgs.append(" 功效 [0,1] 且对 q_true 单调非降")
260             # 置换零分布均值 理论(m-1)/(n-1)
261             pm_ok = True
262             for k, v in res['permutation']['null_samples'].items():
263                 if abs(v['mean'] - v['theory_mean']) > 0.03:
264                     pm_ok = False
265             if pm_ok:
266                 msgs.append(" 置换经验零分布均值 (m-1)/(n-1)")
267             else:
268                 msgs.append(" 置换零分布与理论略有偏差(m/n较大时正常)")
269         return ok, msgs
270
271 def main():
272     print("[问题三] 四探测器检验框架 + 两类错误 ...")
273     dis = u.load_disease()
274     res = {}
275     res['factor'] = factor_detector(dis)
276     print(" 因子探测器:")
277     for r in res['factor']:
278         print(f"      {r['variable']}: F={r['F']} p={r['p_value']:.2e}
279             显著={r['significant']}")
280     res['risk'] = risk_detector(dis, region='')
281     print(f" 风险探测器(region): C(a,2)={res['risk']['n_pairs']} 对, "
282           f" 原始显著{res['risk']['n_sig_raw']},
283           Bonf{res['risk']['n_sig_bonferroni']},
284           f"BH-FDR{res['risk']['n_sig_bh_fdr']},
285           FWER未校正 {res['risk']['fwer_uncorrected_approx']}")
286     res['interaction'] = interaction_detector(dis)
287     print(" 交互探测器:")
288     for p in res['interaction']['pairs']:
289         print(f"      {p['pair']}: q_adj_joint={p['q_adj_joint']} →
290             {p['classification']}")
291     res['ecological'] = ecological_detector(dis)
292     print(" 生态探测器:")
293     for r in res['ecological']:
294         print(f"      {r['pair']}: F={r['F_ratio']} p={r['p_value']}
295             差异显著={r['significant_diff']}")
296     res['power'] = power_curves()
297     res['tradeoff'] = type1_type2_tradeoff()
298     print(f" 两类错误权衡: 最优 m*={res['tradeoff']['m_star']}")
299     res['permutation'] = permutation_demo(dis)
300     print(" 置换经验零分布完成")
301
302     ok, msgs = validate_constraints(res)
303     print("[约束验证]")
304     for mm in msgs:
305         print("      ", mm)
306     res['validation'] = {'passed': ok, 'messages': msgs}
307
308     out = os.path.join(FIG_DIR, 'problem_3_results.json')
309     with open(out, 'w', encoding='utf-8') as f:
310         json.dump(res, f, ensure_ascii=False, indent=2)
311     print(" 已保存:", out)
312     return res
313
314 if __name__ == '__main__':
315     main()

```

1.5 问题四: OPGD 选择性偏差消减

```

1 # -*- coding: utf-8 -*-
2 # __mh_autobootstrap_syspath__
3 import os as _mh_os, sys as _mh_sys
4 _mh_here = _mh_os.path.dirname(_mh_os.path.abspath(__file__))
5 if _mh_here and _mh_here not in _mh_sys.path:
6     _mh_sys.path.insert(0, _mh_here)
7
8 """问题四: OPGD 过拟合的消减方法 + 效益-代价分析。
9 诊断: OPGD 取 max_q 的选择性偏差(胜者诅咒) E[max q]>E[q]。
10 消减: A.K折交叉验证 B.置换+选择性推断 C.调整q惩罚 D.Bootstrap optimism.

```

```

11 输出: figures/publications/results.json
12 """
13 import os, sys
14 _HERE = os.path.dirname(os.path.abspath(__file__))
15 if _HERE not in sys.path:
16     sys.path.insert(0, _HERE)
17
18 import json
19 import numpy as np
20 import utils as u
21
22 FIG_DIR = os.path.join(_HERE, '..', 'figures')
23 os.makedirs(FIG_DIR, exist_ok=True)
24
25
26 def make_search_space(rng, n_params=30, m_range=(3, 10)):
27     """构造 OPGD 参数空间  $\Theta$ : 每个  $\theta$  是一种离散化方案 (用随机切分点模拟
28     '分段方法*分段数*格网边长'的组合)。返回生成离散化标签的函数列表。"""
29     schemes = []
30     for _ in range(n_params):
31         m = rng.integers(m_range[0], m_range[1] + 1)
32         # 用分位数型切分点 (依赖 X 的取值), 模拟不同离散化方案
33         qs = np.sort(rng.uniform(0.05, 0.95, size=m - 1))
34         schemes.append(qs)
35     return schemes
36
37
38 def discretize(x, quantile_cuts):
39     """按分位数切分 x 得分区标签。"""
40     cuts = np.quantile(x, quantile_cuts)
41     return np.digitize(x, cuts)
42
43
44 def opgd_search(x, y, schemes):
45     """OPGD: 遍历  $\Theta$  取 max q。返回 (max_q, best_idx, all_q)。"""
46     qs = np.array([u.q_stat(y, discretize(x, s)) for s in schemes])
47     bi = int(np.argmax(qs))
48     return float(qs[bi]), bi, qs
49
50
51 def winners_curse_sim(n=150, n_params=30, reps=1000, seed=u.SEED):
52     """胜者诅咒仿真: 纯噪声 XY 下, E[max_q] vs E[单个 q]。"""
53     rng = np.random.default_rng(seed)
54     max_qs = np.empty(reps)
55     single_qs = np.empty(reps)
56     for r in range(reps):
57         x = rng.standard_normal(n)
58         y = rng.standard_normal(n) # H0: 与 X 独立
59         schemes = make_search_space(rng, n_params)
60         mq, bi, allq = opgd_search(x, y, schemes)
61         max_qs[r] = mq
62         single_qs[r] = allq.mean() # 平均候选 q (单个参数的典型表现, 无选择)
63     return {'n': n, 'n_params': n_params, 'reps': reps,
64           'E_max_q': round(float(max_qs.mean()), 4),
65           'E_single_q': round(float(single_qs.mean()), 4),
66           'amplification': round(float(max_qs.mean() /
67                                     single_qs.mean(), 1e-6), 2),
68           'max_q_samples': [round(float(x), 4) for x in max_qs[:500]],
69           'single_q_samples': [round(float(x), 4) for x in
70                               single_qs[:500]]}
71
72
73 def winners_curse_vs_space(n=150, sizes=(5, 10, 20, 40, 80), reps=600,
74                             seed=u.SEED):
75     """E[max_q] 随  $|\Theta|$  增长 (ln| $\Theta$ |)。"""
76     rng = np.random.default_rng(seed)
77     out = {'sizes': list(sizes), 'E_max_q': []}
78     for sz in sizes:
79         mqs = np.empty(reps)
80         for r in range(reps):
81             x = rng.standard_normal(n); y = rng.standard_normal(n)
82             schemes = make_search_space(rng, sz)
83             mq, _, _ = opgd_search(x, y, schemes)
84             mqs[r] = mq
85         out['E_max_q'].append(round(float(mqs.mean()), 4))
86     return out
87
88
89 def cv_opgd(x, y, schemes, K=5, seed=u.SEED):
90     """K 折交叉验证: 训练折选 *, 验证折评 q^CV。
91     验证折上对固定分区 (训练折选出) 用调整 q 校正残差有限样本膨胀——
92     分区已固定, q_adj 是该分区总体解释力的无偏估计, 从而 q^CV 只反映
93     泛化能力 (扣除选择偏差与每折 m=虚高), 而非验证折自身的 (m-1)/(n_val-1)
94     偏差。"""
95     rng = np.random.default_rng(seed)
96     n = len(y)
97     idx = rng.permutation(n)
98     folds = np.array_split(idx, K)
99     q_vals = []
100     for k in range(K):
101         val_idx = folds[k]
102         tr_idx = np.concatenate([folds[j] for j in range(K) if j != k])
103         # 训练折选 *
104         best_q, best_i = -1, 0
105         for i, s in enumerate(schemes):
106             qq = u.q_stat(y[tr_idx], discretize(x[tr_idx], s))
107             if qq > best_q:
108                 best_q, best_i = qq, i
109         # 验证折评估 (用训练折的分位数切分点, 避免数据泄露)
110         cuts = np.quantile(x[tr_idx], schemes[best_i])
111         g_val = np.digitize(y[val_idx], cuts)
112         m_val = int(np.unique(g_val, size))
113         nv = len(val_idx)
114         q_vals.append(u.q_adj(u.q_stat(y[val_idx], g_val), nv, m_val))
115     return float(np.mean(q_vals))
116
117
118 def permutation_maxq(x, y, schemes, B=1000, seed=u.SEED):
119     """对整个 max_q 流程做置换, 得经验零分布 + 校正 p 值。"""
120     rng = np.random.default_rng(seed)
121     obs, _, _ = opgd_search(x, y, schemes)
122     null = np.empty(B)
123     for b in range(B):
124         yp = rng.permutation(y)
125         null[b], _, _ = opgd_search(x, yp, schemes)
126     p = (1 + np.sum(null >= obs)) / (B + 1)
127     return float(obs), float(np.mean(null)), float(p), null
128
129
130 def bootstrap_optimism(x, y, schemes, B=500, seed=u.SEED):
131     """Bootstrap .632 optimism correction: q_corrected = q_app -
132     optimism。"""
133     rng = np.random.default_rng(seed)
134     n = len(y)
135     q_app, best_i, _ = opgd_search(x, y, schemes) # 全数据表现 q
136     optimisms = []
137     for b in range(B):
138         bidx = rng.integers(0, n, size=n)
139         xb, yb = x[bidx], y[bidx]
140         # 在 bootstrap 样本选 *
141         bq, bi = -1, 0
142         for i, s in enumerate(schemes):
143             qq = u.q_stat(yb, discretize(xb, s))
144             if qq > bq:
145                 bq, bi = qq, i
146         # train(boot) - test(原始全样本) 的乐观度
147         cuts = np.quantile(xb, schemes[bi])
148         q_test = u.q_stat(y, np.digitize(x, cuts))
149         optimisms.append(bq - q_test)
150     optimism = float(np.mean(optimisms))
151     return q_app, optimism, float(q_app - optimism)
152
153
154 def four_method_comparison(seed=u.SEED):
155     """在合成弱信号数据上对比四方法 q。"""
156     rng = np.random.default_rng(seed)
157     n = 300
158     # 合成: X 连续, Y 含弱真实信号 (q_true 0.1)
159     x = rng.standard_normal(n)
160     q_true = 0.10
161     # 真实信号: Y 与 X 的某种分箱相关 (用 3 个真实区间)
162     true_bins = np.digitize(x, np.quantile(x, [0.33, 0.66]))

```

```

158 means = u. fix('signal_means(3, q_true)
159 y = means[true_bins] + rng.normal(0, np.sqrt(1 - q_true), size=n)
160
161 schemes = make_search_space(rng, n_params=30, m_range=(3, 8))
162 max_q, bi, _ = opgd_search(x, y, schemes)
163 m_best = int(np.unique(discretize(x, schemes[bi])).size)
164 q_adj_val = u.q_adj(max_q, n, m_best)
165 q_cv = cv_opgd(x, y, schemes, K=5, seed=seed)
166 _, null_mean, p_perm, null = permutation_maxq(x, y, schemes, B=1000,
167 seed=seed)
168 # 置换校正 q: 扣除经验零分布均值 (选择偏差)
169 q_perm_corrected = max_q - null_mean
170
171 q_app, optimism, q_boot = bootstrap_optimism(x, y, schemes, B=500,
172 seed=seed)
173
174 return {'n': n, 'q_true': q_true, 'm_best': m_best,
175         'naive_max_q': round(max_q, 4),
176         'q_adj': round(q_adj_val, 4),
177         'q_cv': round(q_cv, 4),
178         'q_perm_corrected': round(q_perm_corrected, 4),
179         'perm_null_mean': round(null_mean, 4),
180         'perm_p_value': round(p_perm, 4),
181         'q_bootstrap_632': round(q_boot, 4),
182         'bootstrap_optimism': round(optimism, 4)}
183
184 def cost_benefit_radar():
185     """各方法五维度评分(0-10):
186     偏差消减/方差控制/泛化/计算代价(高=代价小,反向)/功效。
187     供雷达图。计算代价维度数值越大表示代价越小(便于雷达图'越大越好')。"""
188     methods = {
189         '朴素 max q': {'偏差消减': 1, '方差控制': 3, '泛化': 2,
190                        '计算经济性': 10, '功效': 9},
191         '调整 q': {'偏差消减': 5, '方差控制': 6, '泛化': 5,
192                    '计算经济性': 9, '功效': 8},
193         'K折交叉验证': {'偏差消减': 9, '方差控制': 7, '泛化': 9,
194                           '计算经济性': 5, '功效': 6},
195         '置换检验': {'偏差消减': 8, '方差控制': 8, '泛化': 8,
196                      '计算经济性': 3, '功效': 8},
197         'Bootstrap .632': {'偏差消减': 8, '方差控制': 7, '泛化': 8,
198                             '计算经济性': 3, '功效': 6},
199     }
200
201     return {'dimensions': ['偏差消减', '方差控制', '泛化', '计算经济性',
202                             '功效'],
203            'methods': methods}
204
205 def cost_benefit_table():
206     """效益-代价对比表(对应 TABLE_cost_benefit)。"""
207     return [
208         {'method': '朴素 max_q', 'bias_reduction': '无(基准)',
209          'generalization': '差',
210          'compute_cost': '低', 'power_loss': '无', 'need_holdout': '否',
211          'complexity': '低'},
212         {'method': '调整 q', 'bias_reduction': '中', 'generalization': '中',
213          'compute_cost': '低', 'power_loss': '低', 'need_holdout': '否',
214          'complexity': '低'},
215         {'method': 'K 折交叉验证', 'bias_reduction': '高',
216          'generalization': '高',
217          'compute_cost': '中(K)', 'power_loss': '中', 'need_holdout':
218          '是', 'complexity': '中'},
219         {'method': '置换检验', 'bias_reduction': '高(p值层面)',
220          'generalization': '高',
221          'compute_cost': '高(B×|Θ|)', 'power_loss': '低', 'need_holdout':
222          '否', 'complexity': '中'},
223         {'method': 'Bootstrap .632', 'bias_reduction': '高',
224          'generalization': '高',
225          'compute_cost': '高(B)', 'power_loss': '中', 'need_holdout':
226          '否(重采样)', 'complexity': '高'},
227     ]
228
229 def validate_constraints(res):
230     ok = True; msgs = []
231     wc = res['winners_curse']
232     # max q >= single q (极大值定义)
233     if wc['E_max_q'] < wc['E_single_q']:
234         ok = False; msgs.append(" E[max q] < E[single q] 违反极大值")
235     else:
236         msgs.append(f" 胜者诅咒: E[max q]={wc['E_max_q']} >>
237                     E[q]={wc['E_single_q']} (放大{wc['amplification']}x)")
238     # max q 随 |Θ| 单调非降
239     em = res['winners_curse_vs_space']['E_max_q']
240     if not all(em[i+1] >= em[i] - 0.01 for i in range(len(em)-1)):
241         msgs.append(" E[max q] 随 |Θ| 非严格单调(随机波动)")
242     else:
243         msgs.append(" E[max q] 随 |Θ| 增长(选择空间越大偏差越重)")
244     # 四方法: 消减后 q < 朴素 max q (退化检测: q^CV 应明显下降)
245     cmp = res['four_method']
246     if cmp['q_cv'] >= cmp['naive_max_q']:
247         ok = False; msgs.append(" q^CV 未低于 max q (CV 实现可能数据泄露)")
248     else:
249         msgs.append(f" q^CV={cmp['q_cv']} < 朴素
250                     max_q={cmp['naive_max_q']} (消除乐观偏差)")
251     # 所有 q 量上合理范围 / CV/置换校正可为负, 但不应>1)
252     for k in ['naive_max_q', 'q_adj', 'q_cv', 'q_perm_corrected',
253              'q_bootstrap_632']:
254         if cmp[k] > 1 + 1e-9:
255             ok = False; msgs.append(f" {k}={cmp[k]} >1 不可能")
256     if ok:
257         msgs.append(" 所有 q 估计 OK")
258     # p 值 [0,1]
259     if not (0 <= cmp['perm_p_value'] <= 1):
260         ok = False; msgs.append(" 置换 p 值越界")
261     else:
262         msgs.append(f" 置换检验 p={cmp['perm_p_value']} [0,1]")
263     return ok, msgs
264
265 def main():
266     print("[问题四] OPGD 胜者诅咒诊断 + 四方法消减 ...")
267     res = {}
268     print(" 胜者诅咒仿真(n=150,|Θ|=30,1000次)...")
269     res['winners_curse'] = winners_curse_sim()
270     wc = res['winners_curse']
271     print(f" E[max q]={wc['E_max_q']} vs E[q]={wc['E_single_q']}
272           放大{wc['amplification']}x")
273     print(" E[max q] vs |Θ| ...")
274     res['winners_curse_vs_space'] = winners_curse_vs_space()
275     print(f" {res['winners_curse_vs_space']['sizes']} ->
276           {res['winners_curse_vs_space']['E_max_q']}")
277     print(" 四方法对比(合成弱信号 q_true=0.10)...")
278     res['four_method'] = four_method_comparison()
279     cmp = res['four_method']
280     print(f" 朴素max_q={cmp['naive_max_q']} 调整q={cmp['q_adj']} "
281           f"q^CV={cmp['q_cv']} 置换校正={cmp['q_perm_corrected']} "
282           f"Boot.632={cmp['q_bootstrap_632']} (q_true={cmp['q_true']})")
283     res['cost_benefit_radar'] = cost_benefit_radar()
284     res['cost_benefit_table'] = cost_benefit_table()
285
286     ok, msgs = validate_constraints(res)
287     print("[约束验证]")
288     for mm in msgs:
289         print(" ", mm)
290     res['validation'] = {'passed': ok, 'messages': msgs}
291
292     out = os.path.join(FIG_DIR, 'problem_4_results.json')
293     with open(out, 'w', encoding='utf-8') as f:
294         json.dump(res, f, ensure_ascii=False, indent=2)
295     print(" 已保存:", out)
296     return res
297
298 if __name__ == '__main__':
299     main()

```

1.6 灵敏度分析

```
1  # -*- coding: utf-8 -*- 73
2  # __mh_autobootstrap_syspath__ 74
3  import os as _mh_os, sys as _mh_sys 75
4  _mh_here = _mh_os.path.dirname(_mh_os.path.abspath(__file__)) 76
5  if _mh_here and _mh_here not in _mh_sys.path: 77
6      _mh_sys.path.insert(0, _mh_here) 78
7  79
8  """灵敏度分析: 关键参数(m, n, q_true, alpha, |θ|)对模型性能的影响。 80
9  输出: figures/sensitivity_results.json (供 paper-figure 81
      画龙卷风图/灵敏度曲线)。 82
10  """ 83
11  import os, sys 84
12  _HERE = os.path.dirname(os.path.abspath(__file__)) 85
13  if _HERE not in sys.path: 86
14      sys.path.insert(0, _HERE) 87
15  88
16  import json 89
17  import numpy as np 90
18  import utils as u 91
19  92
20  FIG_DIR = os.path.join(_HERE, '..', 'figures') 93
21  os.makedirs(FIG_DIR, exist_ok=True) 94
22  95
23  96
24  def sens_m(n=150, q_true=0.1, m_range=None): 97
25      """过拟合强度 & 功效 对 m。""" 98
26      if m_range is None: 99
27          m_range = list(range(2, 41)) 100
28      overfit = [(1 - q_true) * (m - 1) / (n - 1) for m in m_range] 101
29      power = [u.power_ncf(q_true, n, m, 0.05) for m in m_range] 102
30      return {'values': list(m_range), 103
31              'overfit': [round(x, 4) for x in overfit], 104
32              'power': [round(x, 4) for x in power]} 105
33  106
34  107
35  def sens_n(m=8, q_true=0.1, n_range=None): 108
36      """过拟合强度 & 功效 对 n。""" 109
37      if n_range is None: 110
38          n_range = list(range(20, 401, 20)) 111
39      overfit = [(1 - q_true) * (m - 1) / (nn - 1) for nn in n_range] 112
40      power = [u.power_ncf(q_true, nn, m, 0.05) for nn in n_range] 113
41      return {'values': list(n_range), 114
42              'overfit': [round(x, 4) for x in overfit], 115
43              'power': [round(x, 4) for x in power]} 116
44  117
45  118
46  def sens_qtrue(n=150, m=8, q_range=None): 119
47      """过拟合强度 & 功效 对 q_true。""" 120
48      if q_range is None: 121
49          q_range = [round(x, 3) for x in np.linspace(0, 0.8, 21)] 122
50      overfit = [(1 - qt) * (m - 1) / (n - 1) for qt in q_range] 123
51      power = [u.power_ncf(qt, n, m, 0.05) for qt in q_range] 124
52      return {'values': q_range, 125
53              'overfit': [round(x, 4) for x in overfit], 126
54              'power': [round(x, 4) for x in power]} 127
55  128
56  129
57  def sens_alpha(n=150, m=8, q_true=0.1, alpha_range=None): 130
58      """两类错误对 alpha。""" 131
59      if alpha_range is None: 132
60          alpha_range = [0.001, 0.005, 0.01, 0.025, 0.05, 0.1, 0.15, 0.2] 133
61      power = [u.power_ncf(q_true, n, m, a) for a in alpha_range] 134
62      beta = [1 - p for p in power] 135
63      return {'values': alpha_range, 136
64              'alpha': alpha_range, 137
65              'beta': [round(b, 4) for b in beta], 138
66              'power': [round(p, 4) for p in power]} 139
67  140
68  141
69  def sens_searchspace(n=150, sizes=None, reps=400, seed=u.SEED): 142
70      """胜者诅咒偏差对 |θ| (应 ln|θ|)。""" 143
71      import problem4 as p4 144
72      if sizes is None: 145
73          sizes = [2, 5, 10, 20, 40, 80, 160] 146
74          rng = np.random.default_rng(seed) 147
75          emax = [] 148
76          for sz in sizes: 149
77              mqs = np.empty(reps) 150
78              for r in range(reps): 151
79                  x = rng.standard_normal(n); y = rng.standard_normal(n) 152
79                  sch = p4.make_search_space(rng, sz) 153
80                  mq, _, _ = p4.opgd_search(x, y, sch) 154
81                  mqs[r] = mq 155
82                  emax.append(round(float(mqs.mean()), 4)) 156
83          return {'values': list(sizes), 'E_max_q': emax, 157
84                  'log_sizes': [round(float(np.log(s)), 4) for s in sizes]} 158
85  159
86  def tornado(n=150, m=8, q_true=0.1): 160
87      """龙卷风图: 各参数 ±20% 对过拟合强度的影响。""" 161
88      base = (1 - q_true) * (m - 1) / (n - 1) 162
89      rows = [] 163
90      for name, val, fn in [ 164
91          ('m', m, lambda v: (1 - q_true) * (v - 1) / (n - 1)), 165
92          ('n', n, lambda v: (1 - q_true) * (m - 1) / (v - 1)), 166
93          ('q_true', q_true, lambda v: (1 - v) * (m - 1) / (n - 1)), 167
94      ]: 168
95          lo = fn(val * 0.8) 169
96          hi = fn(val * 1.2) 170
97          rows.append({'parameter': name, 'base': round(base, 4), 171
98                      'low': round(lo, 4), 'high': round(hi, 4), 172
99                      'range': round(abs(hi - lo), 4)}) 173
100      rows.sort(key=lambda r: r['range'], reverse=True) 174
101      return {'baseline': round(base, 4), 'parameters': rows} 175
102  176
103  def main(): 177
104      print("[灵敏度分析] m / n / q_true / alpha / |θ| ...") 178
105      res = {} 179
106      res['vs_m'] = sens_m() 180
107      res['vs_n'] = sens_n() 181
108      res['vs_qtrue'] = sens_qtrue() 182
109      res['vs_alpha'] = sens_alpha() 183
110      print(" 搜索空间 |θ| 灵敏度仿真...") 184
111      res['vs_searchspace'] = sens_searchspace() 185
112      res['tornado'] = tornado() 186
113      print(" 龙卷风图(±20%)排序:", [r['parameter'] for r in 187
114          res['tornado']['parameters']]) 188
115  189
116      # 验证方向性 190
117      msgs = [] 191
118      ov_m = res['vs_m']['overfit'] 192
119      msgs.append(" Overfit 随 m 单调增" if all(ov_m[i+1] >= ov_m[i] for i 193
120          in range(len(ov_m)-1)) 194
121                  else " Overfit-m 方向错") 195
122      ov_n = res['vs_n']['overfit'] 196
123      msgs.append(" Overfit 随 n 单调减" if all(ov_n[i+1] <= ov_n[i]+1e-9 197
124          for i in range(len(ov_n)-1)) 198
125                  else " Overfit-n 方向错") 199
126      pw = res['vs_qtrue']['power'] 200
127      msgs.append(" 功效随 q_true 单调增" if all(pw[i+1] >= pw[i]-1e-6 for i 201
128          in range(len(pw)-1)) 202
129                  else " 功效-q_true 方向错") 203
124      em = res['vs_searchspace']['E_max_q'] 204
125      msgs.append(" E[max q] 随 |θ| 增长" if em[-1] > em[0] 206
126                  else " E[max q]-|θ| 方向错") 207
127      print("[灵敏度方向性验证]") 208
128      for mm in msgs: 209
129          print(" ", mm) 210
130      res['validation'] = {'messages': msgs} 211
131  212
132      out = os.path.join(FIG_DIR, 'sensitivity_results.json') 213
133      with open(out, 'w', encoding='utf-8') as f: 214
134          json.dump(res, f, ensure_ascii=False, indent=2) 215
135      print(" 已保存:", out) 216
136      return res 217
```

142
143

144
145

```
if __name__ == '__main__':  
    main()
```

1.7 主程序入口 main.py

```
1  # -*- coding: utf-8 -*-  
2  # _mh_autobootstrap_syspath_  
3  import os as _mh_os, sys as _mh_sys  
4  _mh_here = _mh_os.path.dirname(_mh_os.path.abspath(__file__))  
5  if _mh_here and _mh_here not in _mh_sys.path:  
6      _mh_sys.path.insert(0, _mh_here)  
7  
8  """主入口：依次运行问题一-四 + 灵敏度分析，汇总到 figures/all_results.json。  
9  地理探测器 q 统计量的过拟合机理、检验框架与消减方法。  
10 seed=42, 蒙特卡洛/置换 reps=B=2000, 交叉验证 K=5。  
11 """  
12 import os, sys  
13 _HERE = os.path.dirname(os.path.abspath(__file__))  
14 if _HERE not in sys.path:  
15     sys.path.insert(0, _HERE)  
16  
17 import json  
18 import problem1  
19 import problem2  
20 import problem3  
21 import problem4  
22 import sensitivity_analysis as sens  
23  
24 FIG_DIR = os.path.join(_HERE, '..', 'figures')  
25 os.makedirs(FIG_DIR, exist_ok=True)  
26  
27  
28 def main():  
29     print("=" * 60)  
30     print("地理探测器 q 统计量：过拟合机理 / 检验框架 / 消减方法")  
31     print("seed=42  reps=B=2000  K=5")  
32     print("=" * 60)  
33  
34     all_res = {}  
35  
36     print("\n>>> 问题一：q 的统计本质与零假设期望")  
37     all_res['problem1'] = problem1.main()  
38
```

```
39     print("\n>>> 问题二：过拟合发生条件与强度")  
40     all_res['problem2'] = problem2.main()  
41  
42     print("\n>>> 问题三：四探测器统一检验框架 + 两类错误")  
43     all_res['problem3'] = problem3.main()  
44  
45     print("\n>>> 问题四：OPGD 胜者诅咒诊断 + 四方法消减")  
46     all_res['problem4'] = problem4.main()  
47  
48     print("\n>>> 灵敏度分析")  
49     all_res['sensitivity'] = sens.main()  
50  
51     # 汇总各子问题验证状态  
52     summary = {}  
53     for k in ['problem1', 'problem2', 'problem3', 'problem4']:  
54         v = all_res[k].get('validation', {})  
55         summary[k] = {'passed': v.get('passed', None),  
56                     'n_messages': len(v.get('messages', []))}  
57     summary['sensitivity'] = {'messages': all_res['sensitivity']  
58                             .get('validation', {}).get('messages', [])}  
59     all_res['_summary'] = summary  
60  
61     out = os.path.join(FIG_DIR, 'all_results.json')  
62     with open(out, 'w', encoding='utf-8') as f:  
63         json.dump(all_res, f, ensure_ascii=False, indent=2)  
64  
65     print("\n" + "=" * 60)  
66     print("[汇总] 各子问题约束验证：")  
67     for k in ['problem1', 'problem2', 'problem3', 'problem4']:  
68         print(f"    {k}: passed={summary[k]['passed']}")  
69     print("  已保存汇总:", out)  
70     print("=" * 60)  
71     return all_res  
72  
73  
74 if __name__ == '__main__':  
75     main()
```